



universidad
de león



Departamento de Ingenierías
Mecánica, Informática y Aeroespacial

MÁSTER UNIVERSITARIO EN ROBÓTICA E INTELIGENCIA ARTIFICIAL

Visión por Computador

CLASIFICACIÓN DE TEXTURAS

Carlos González Carballo.

Repositorio Github: [Carlos.gcg00/Vision por Computador](https://github.com/Carlos.gcg00/Vision_por_Computador)

Índice general

Índice de figuras	III
Índice de tablas	V
1. Introducción	1
2. Análisis del dataset.	2
2.1. Filtrar datos con DBSCAN	5
2.2. Eliminación de outliers	5
3. Metodología de entrenamiento	8
3.1. Modelos seleccionados	10
3.1.1. Construcción de los modelos	12
3.2. Estrategia de entrenamiento	12
3.2.1. <i>Callbacks</i>	13
3.3. Configuración del entrenamiento	13
3.3.1. Optimizadores	13
3.3.2. Función de pérdida	13
3.4. Métricas de evaluación	14
4. Resultados del entrenamiento	16
4.1. Escenario 1: Entrenamiento sobre el dataset reducido y filtrado	17
4.1.1. Experimento 1: VGG16 con Adam y Batch Size 8	18
4.1.2. Experimento 2: VGG16 con SGD y Batch Size 8	21
4.1.3. Experimento 3: VGG16 con Adam y Batch Size 64	24
4.1.4. Experimento 4: EfficientNetB0 con Adam y Batch Size 64 . .	27
4.1.5. Experimento 5: EfficientNetB3 con Adam y Batch Size 64 . .	30
4.1.6. Experimento 6: VGG19 con Adam y Batch Size 64	33
4.2. Escenario 2: Entrenamiento sobre el dataset filtrado con 15 clases . .	36

4.2.1.	Experimento 1: VGG16 con Adam y Batch Size 64	36
4.2.2.	Experimento 2: EfficientNetB3 con Adam y Batch Size 64 . . .	40
4.3.	Escenario 3: Entrenamiento sobre el dataset reducido sin filtrar	43
4.3.1.	Experimento 1: VGG16 con Adam y Batch Size 64	43
4.3.2.	Experimento 2: EfficientNetB0 con Adam y Batch Size 64 . . .	46
5.	Conclusiones	49
	Bibliografía	52

Índice de figuras

2.1. Distribución de los datos	2
2.2. Ejemplos de imágenes de entrenamiento	3
2.3. Ejemplos de imágenes de test	3
2.4. <i>Scatterplot</i> , anchos vs alto de las imágenes	4
2.5. DBSCAN	5
2.6. Eliminación outliers con limites superiores e inferiores	6
2.7. Eliminación outliers con limites superiores (+5 %) e inferiores (−5 %)	7
3.1. Distribución de los datos	10
3.2. Arquitectura VGG16 [6]	11
3.3. Arquitectura EfficientnetB0 [2]	11
4.1. Evolución del entrenamiento durante las fases de <i>transfer learning</i> y <i>fine-tuning</i> — Escenario 1, Experimento 1	19
4.2. Matriz de confusión en el conjunto de test — Escenario 1, Experimento 1	20
4.3. Evolución del entrenamiento durante las fases de <i>transfer learning</i> y <i>fine-tuning</i> — Escenario 1, Experimento 2	22
4.4. Matriz de confusión en el conjunto de test — Escenario 1, Experimento 2	23
4.5. Evolución del entrenamiento durante las fases de <i>transfer learning</i> y <i>fine-tuning</i> — Escenario 1, Experimento 3	25
4.6. Matriz de confusión en el conjunto de test — Escenario 1, Experimento 3	26
4.7. Evolución del entrenamiento durante las fases de <i>transfer learning</i> y <i>fine-tuning</i> — Escenario 1, Experimento 4	28
4.8. Matriz de confusión en el conjunto de test — Escenario 1, Experimento 4	29

4.9. Evolución del entrenamiento durante las fases de <i>transfer learning</i> y <i>fine-tuning</i> — Escenario 1, Experimento 5	31
4.10. Matriz de confusión en el conjunto de test — Escenario 1, Experimento 5	32
4.11. Evolución del entrenamiento durante las fases de <i>transfer learning</i> y <i>fine-tuning</i> — Escenario 1, Experimento 6	34
4.12. Matriz de confusión en el conjunto de test — Escenario 1, Experimento 6	35
4.13. Evolución del entrenamiento durante las fases de <i>transfer learning</i> y <i>fine-tuning</i> — Escenario 2, Experimento 1	38
4.14. Matriz de confusión — Escenario 2, Experimento 1	39
4.15. Evolución del entrenamiento durante las fases de <i>transfer learning</i> y <i>fine-tuning</i> — Escenario 2, Experimento 2	41
4.16. Matriz de confusión — Escenario 2, Experimento 2	42
4.17. Evolución del entrenamiento durante las fases de <i>transfer learning</i> y <i>fine-tuning</i> — Experimento 1	44
4.18. Matriz de confusión — Escenario 3, Experimento 1	45
4.19. Evolución del entrenamiento durante las fases de <i>transfer learning</i> y <i>fine-tuning</i> — Escenario 3, Experimento 2	47
4.20. Matriz de confusión — Escenario 3, Experimento 2	48

Índice de tablas

3.1. Número de muestras en cada conjunto tras las distintas fases de pre-procesado	9
4.1. Asignación de índices (0–14) a etiquetas de clase.	17
4.2. Configuración del Experimento 1 del Escenario 1	18
4.3. Resultados finales del Escenario 1, Experimento 1	20
4.4. Configuración del Experimento 2 del Escenario 1	21
4.5. Resultados finales del Escenario 1, Experimento 2	23
4.6. Configuración del Experimento 3 del Escenario 1	24
4.7. Resultados finales del Escenario 1, Experimento 3	26
4.8. Configuración del Experimento 4 del Escenario 1	27
4.9. Resultados finales del Escenario 1, Experimento 4	29
4.10. Configuración del Experimento 5 del Escenario 1	30
4.11. Resultados finales del Escenario 1, Experimento 5	32
4.12. Configuración del Experimento 6 del Escenario 1	33
4.13. Resultados finales del Escenario 1, Experimento 6	35
4.14. Configuración del Experimento 1 del Escenario 2	37
4.15. Resultados finales del Escenario 2, Experimento 1	39
4.16. Configuración del Experimento 2 del Escenario 2	40
4.17. Resultados finales del Escenario 2, Experimento 2	42
4.18. Configuración del Experimento 1 del Escenario 3	43
4.19. Resultados finales del Escenario 3, Experimento 1	45
4.20. Configuración del Experimento 2 del Escenario 3	46
4.21. Resultados finales del Escenario 3, Experimento 2	48

Capítulo 1

Introducción

Este proyecto resuelve la tercera práctica propuesta en la asignatura de Visión por Computadora del máster MURIA. En esta práctica se pretende realizar el entrenamiento de un modelo preentrenado sobre un dataset libre a escoger dentro de la plataforma de HuggingFace [1].

El dataset que se escogió finalmente es un dataset publicado por el usuario Bingsu con el nombre ***Human Action Recognition*** [3]. Es un dataset que recopila 18.000 imágenes (que se dividen en el subconjunto de entrenamiento y evaluación) y tiene 15 clases diferentes. En el Capítulo 2 se detallará las características del dataset, pero es un dataset que resulta muy importante estudiar los datos y filtrarlos.

En la práctica se exige probar al menos tres configuraciones diferentes. Pero ante el problema que hay que resolver se proponen tres estudios diferentes donde se probarán en cada situación varias configuraciones, intentando buscar alguna que ofrezca buenos resultados. El primer estudio consistirá en emplear no solo los datos filtrados, si no, limitar el estudio a 5 clases, en vez de 15. El segundo estudio consistirá en emplear los datos filtrados y las 15 clases. Y por último, el tercer estudio consistirá en emplear los datos originales.

Capítulo 2

Análisis del dataset.

El dataset de *Human Action Recognition* contiene dos subconjuntos de datos:

- "*train*": datos que se espera utilizar como conjunto de datos de entrenamiento. Tiene un total de 12.600 imágenes.
- "*test*": datos que se espera utilizar como conjunto de datos de validación. Tiene un total de 5.400 imágenes.

AL tener 15 clases objetivo el primer paso es observar el balanceo del mismo sobre los dos conjuntos. La Figura 2.1 muestra la distribución de las clases en ambos conjuntos, los datos de *train* están balanceados mientras que el conjunto de *test* solo se registra una clase "*calling*". Para poder evaluar bien un modelo necesitamos que este contenga una representación equitativa como en los datos de entrenamiento.

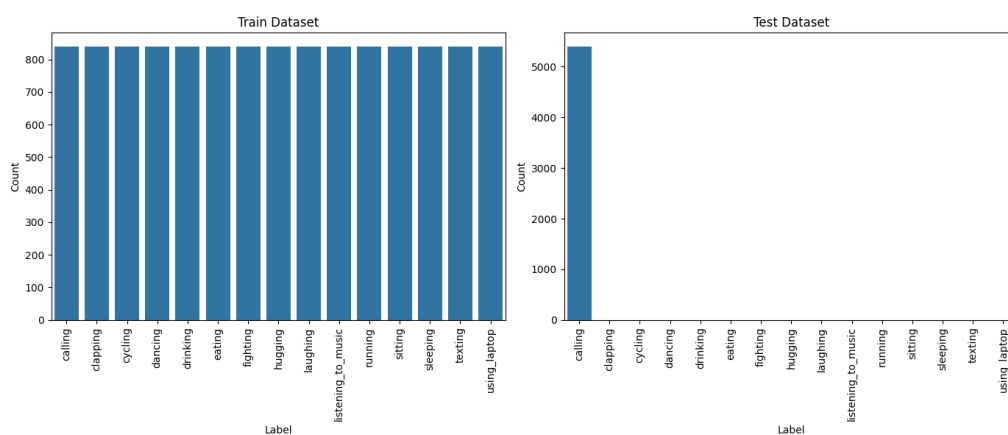


Figura 2.1: Distribución de los datos

La Figura 2.2 y 2.3 son ejemplos de datos de entrenamiento y test, respectivamente. En la segunda imagen se observa que las etiquetas no tienen correlación con las acciones de las imágenes; están mal etiquetadas.



Figura 2.2: Ejemplos de imágenes de entrenamiento

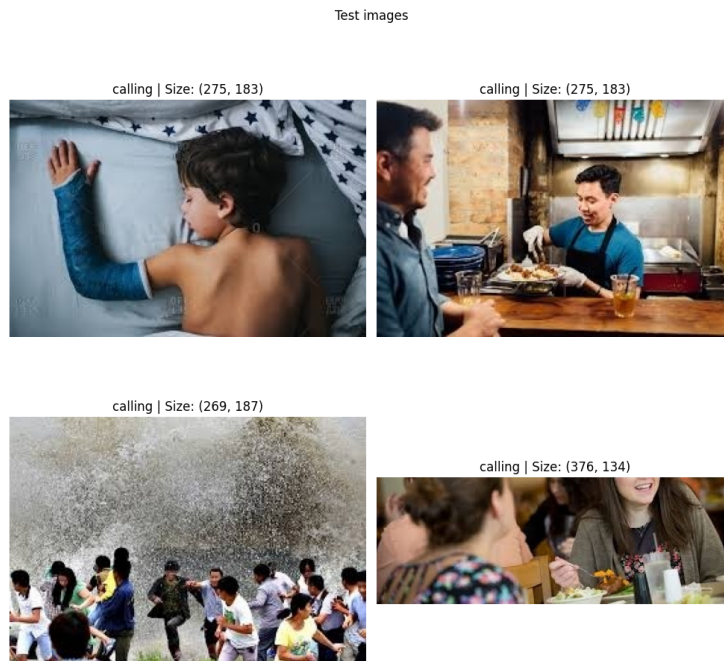


Figura 2.3: Ejemplos de imágenes de test

Ante este dilema se renombró este subconjunto a "*noise*" de esta manera se quitan estos datos del objetivo del problema, pero sin eliminarlos. Este paso tiene

sentido en un proyecto profesional donde en vez de eliminar esos datos, se emplea el modelo para ayudar a etiquetar los datos. Es decir, con un buen modelo se podría realizar un etiquetado semisupervisado.

Como paso previo a dividir los datos es necesario estudiar el tamaño de las imágenes. Al observar que el dataset contenía una gran variedad de imágenes y con tamaños de imágenes distintas se graficó en formato de nube de puntos el ancho y alto de estas en función de su etiqueta, ver Figura 2.4. En la imagen se observa muchos valores atípicos que se alejan mucho de la tendencia del "cuarto de luna", de hecho, los valores más alejados son los que presentan una menor resolución.

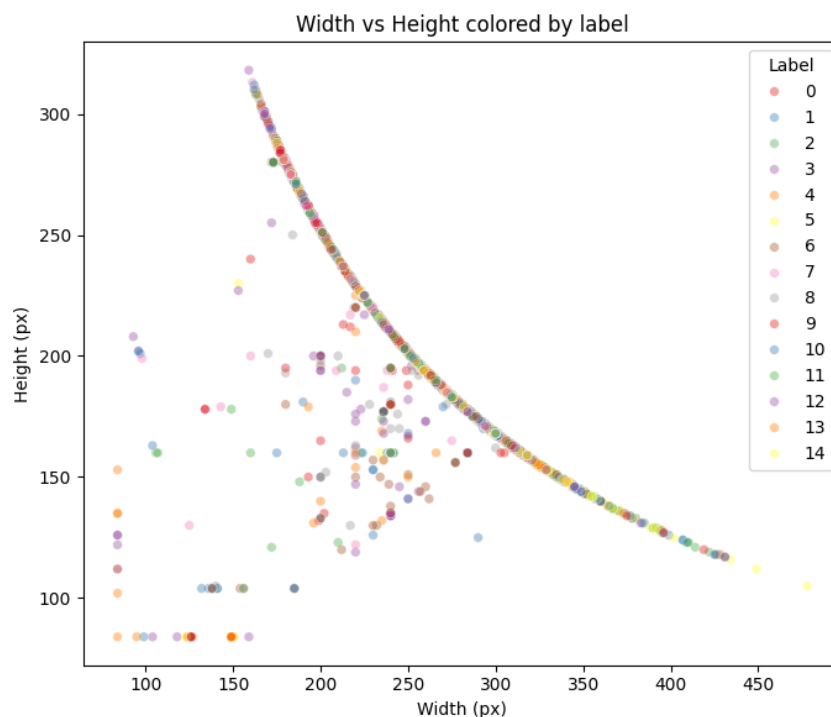


Figura 2.4: *Scatterplot*, anchos vs alto de las imágenes

Se genera la hipótesis de que el modelo mantendrá mejores resultados si las imágenes mantienen una resolución similar, ya que así, al reescalar, estas no pierden tanta información y el modelo no aprende patrones no deseados. Existen varias maneras de filtrar los datos, pero se estudiaron varias opciones:

- Filtrar los datos empleando DBSCAN ya que este algoritmo permite la detección de outliers.
- Eliminando aquellos valores que se encuentren fuera de un umbral

2.1. Filtrar datos con DBSCAN

DBSCAN es un algoritmo de clustering basado en densidad [4]. Agrupa los datos que están cerca entre si dentro de un radio determinado (eps) e indicando el número mínimo de datos que tienen que estar dentro de ese radio ($min_samples$, número mínimo de vecinos).

En este caso, el primer paso es estandarizar los datos de ancho y alto de las imágenes. Y tras realizar varias pruebas, se obtuvo un buen resultado empleando un $eps=0.30$ y $min_samples=20$. Empleando esta configuración se obtuvieron los resultados que se muestran en la Figura 2.5. El primer grafico son los clusters generados y los outliers. El segundo es filtrando los clusters eliminando solo los outliers. Y el tercero es seleccionando solo el "Cluster 1" ya que practicamente engloba el "cuarto de luna" al completo.

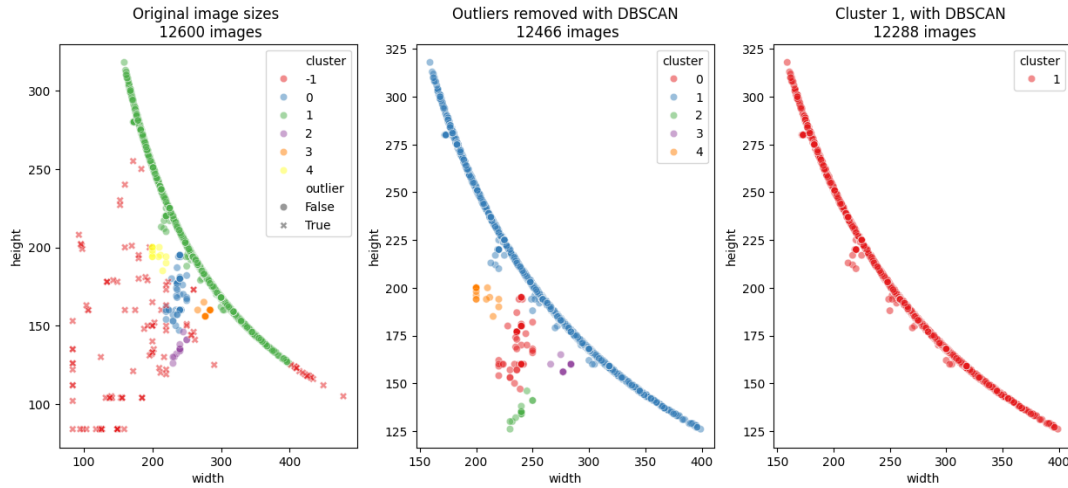


Figura 2.5: DBSCAN

Seleccionando solo los datos del cluster 1, solo se eliminan 312 imágenes del dataset, lo que permite mantener la mayoría de los datos.

2.2. Eliminación de outliers

Conociendo el ancho y alto de las imágenes, se puede estimar la resolución de las mismas.

Para cada una de estas tres variables se calculan los rangos intercuartílicos ($Q1$ y $Q3$) y con ellos se pueden calcular los limites inferiores y superiores que permiten obtener los *outliers*.

- $Q1$ (primer cuartil): valor debajo del cual se encuentra el 25 % de los datos.
- $Q3$ (tercer cuartil): valor debajo del cual se encuentra el 75 % de los datos.
- Rango intercuartílico: $IQR = Q3 - Q1$.
- Límite inferior (*lower*): $lower = Q1 - 1.5 \cdot IQR$
- Límite superior (*upper*): $upper = Q3 + 1.5 \cdot IQR$

Siendo así y tras probar varias combinaciones se optó por considerar que un punto es *outlier* cuando se cumplen las siguientes dos premisas:

1. Es *outlier* si su ancho o alto están fuera de los límites de una de estas dos variables.
2. Es *outlier* si la resolución de la imagen está fuera de los límites de esta variable.

La Figura 2.6 representa los outliers bajo estas condiciones. Si se aplica este método se elimina 708 imágenes, pero además se observa la eliminación de muestras que se encuentran en el "cuarto de luna" que son muestras que se esperan que sean válidas.

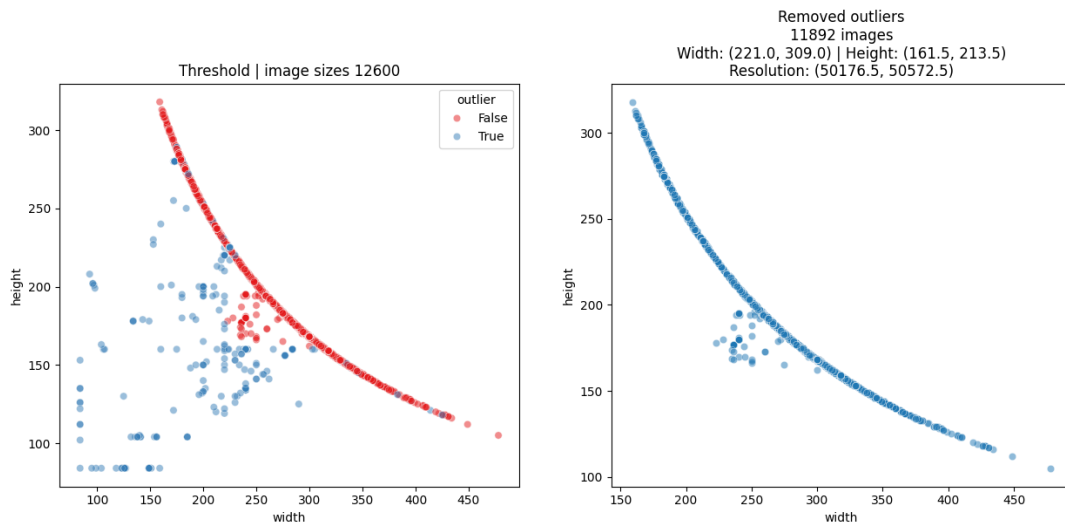


Figura 2.6: Eliminación outliers con límites superiores e inferiores

Con el fin de corregir estos Falsos Negativos se decide aumentar y disminuir en un 5 % los límites superiores e inferiores respectivamente. Tras esta modificación se elimina solo 152 imágenes y se obtiene una mejor distribución (ver Figura 2.7).

Tras comparar las alternativas para homogeneizar el dataset (variabilidad de tamaños y resoluciones), se selecciona el método que mejor equilibra calidad de las muestras, interpretabilidad y preservación de datos.

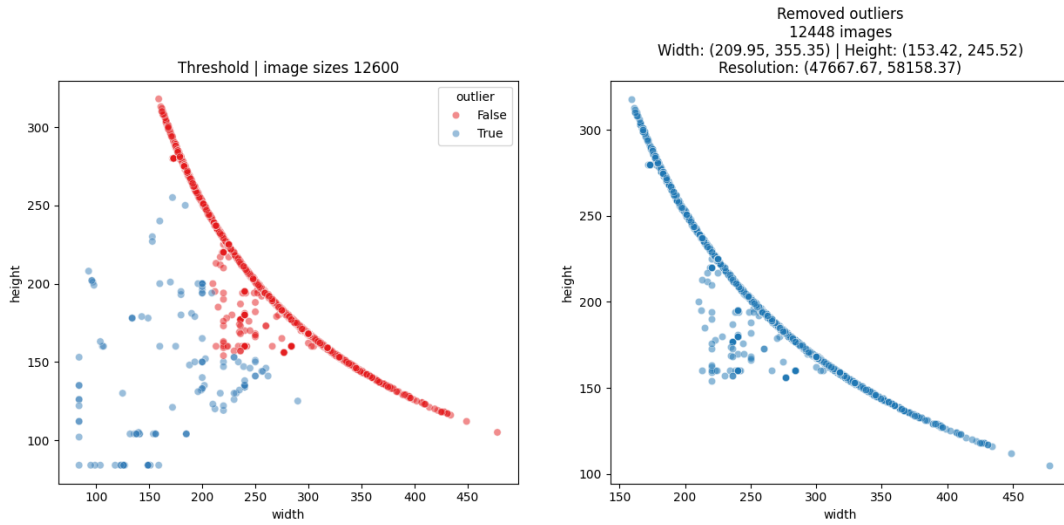


Figura 2.7: eliminación outliers con límites superiores (+5 %) e inferiores (−5 %)

- DBSCAN: permite una segmentación razonable por densidad, pero el resultado depende de la búsqueda de hiperparámetros (eps , $min_samples$). Además, obliga a decidir a posteriori qué clusters se conservan, lo que reduce la reproducibilidad del pipeline y hace más difícil justificar el criterio de filtrado.
- Eliminación de outliers con el IQR: elimina una imagen únicamente si es *outlier* a la vez en dimensiones y en resolución. Este enfoque es el más equilibrado porque reduce falsos positivos (no elimina tantas muestras) y mantiene un criterio determinista y fácilmente interpretable.

Por tanto, finalmente se escogió el segundo criterio ya que es más interpretable y reproducible, además, mantiene el 98.79 % de los datos frente al 97.52 % con DBSCAN, es una mejora ligera pero suficiente.

Capítulo 3

Metodología de entrenamiento

En este capítulo se describen los fundamentos metodológicos y los criterios técnicos seguidos durante las distintas fases del proceso de entrenamiento de los modelos propuestos. Los resultados derivados de estos experimentos se presentan y analizan en el Capítulo 4.

Como primera etapa, el conjunto de datos original se divide en tres subconjuntos disjuntos: entrenamiento (70

Previamente a la segmentación, se lleva a cabo un análisis exploratorio con el objetivo de evaluar la uniformidad en las dimensiones de las imágenes. A partir de este análisis se define una estrategia de filtrado basada en criterios estadísticos, aplicada de forma independiente a cada uno de los subconjuntos, con el fin de eliminar valores atípicos y preservar la representatividad de los datos.

Dado el elevado número de clases presentes en el conjunto de datos, el proceso de entrenamiento se plantea bajo tres escenarios diferenciados:

- Un conjunto de datos filtrado, en el que las imágenes se seleccionan aplicando un criterio basado en el rango intercuartílico, con el objetivo de eliminar valores atípicos en términos de tamaño.
- Un conjunto de datos reducido y filtrado, obtenido a partir del conjunto filtrado, que considera únicamente un subconjunto de cinco clases. Este escenario plantea un problema de menor complejidad y se espera que facilite el aprendizaje del modelo, por lo que constituye el principal foco de interés en el proceso de entrenamiento.

- Un conjunto de datos reducido sin filtrado previo, obtenido directamente a partir del conjunto original, que considera el mismo subconjunto de cinco clases y permite evaluar el impacto del filtrado en presencia de imágenes no normalizadas en tamaño.

Este planteamiento permite realizar diversas comparativas de interés. En primer lugar, se analiza el impacto del filtrado de datos comparando el conjunto completo original con el conjunto completo filtrado. En segundo lugar, se comparan los dos conjuntos reducidos, uno filtrado y otro no filtrado, lo que permite evaluar de forma aislada el efecto del preprocesado sobre un problema de clasificación de menor complejidad. Dado que el conjunto reducido y filtrado presenta un escenario más controlado y equilibrado, se espera que los modelos alcancen su mejor rendimiento en este caso, motivo por el cual se pone especial énfasis en su análisis.

La Tabla 3.1 muestra la distribución del número de muestras en cada uno de los escenarios considerados, mientras que la Figura 3.1 recoge la distribución de clases para los conjuntos original y filtrado. En los casos reducidos, se han seleccionado las siguientes clases: *calling*, *clapping*, *cycling*, *dancing* y *drinking*.

Tabla 3.1: Número de muestras en cada conjunto tras las distintas fases de preprocesado

Conjunto	Original	Reducido	Filtrado	Reducido y Filtrado
Entrenamiento	6173	2940	6099	2034
Validación	1323	630	1305	437
Test	1323	630	1307	435

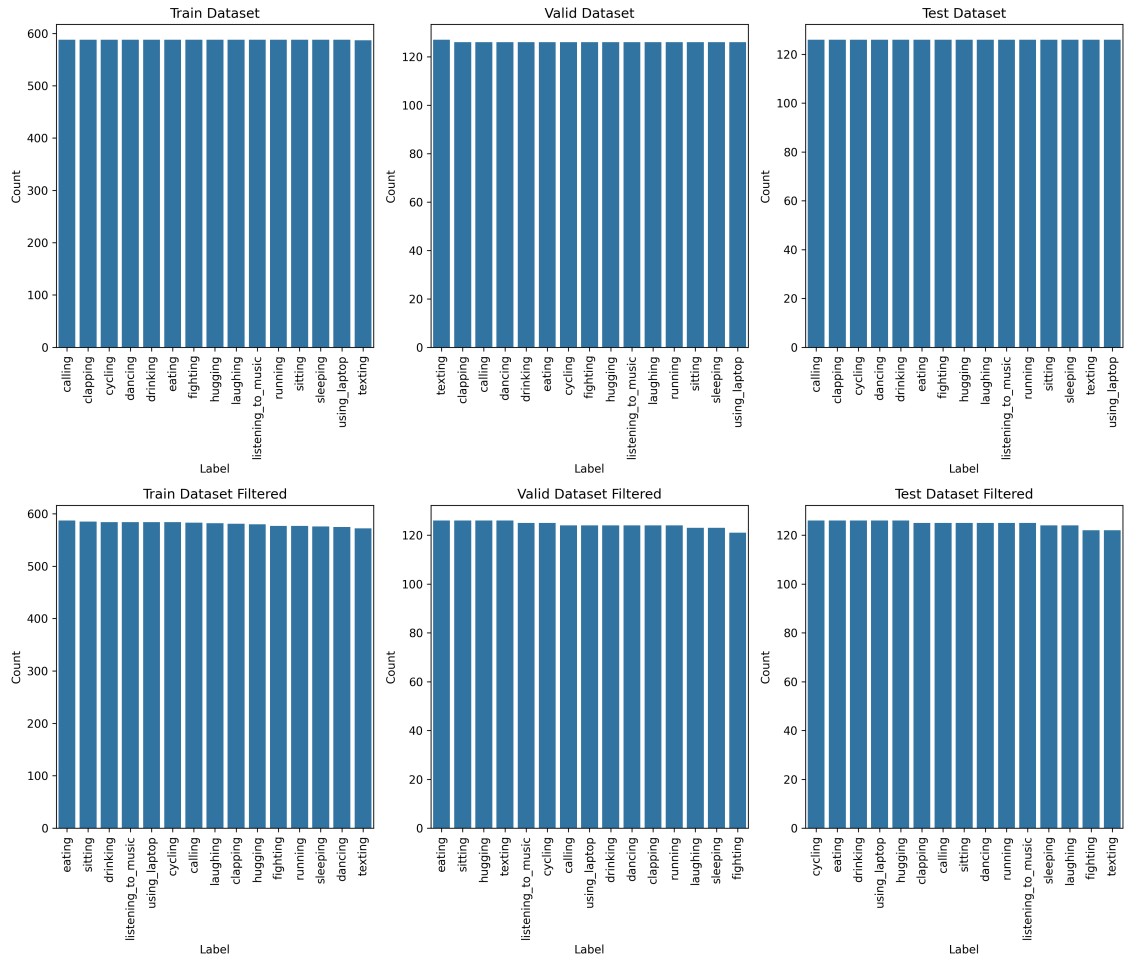


Figura 3.1: Distribución de los datos

3.1. Modelos seleccionados

Los modelos seleccionados para el proceso de entrenamiento pertenecen a dos familias de arquitecturas de redes neuronales convolucionales: **VGG16** y **EfficientNet**, [8], [9]. En particular se emplean los modelos **VGG16**, **VGG19**, **EfficientNetB0** y **EfficientNetB3**, cada familia presenta una arquitectura diferente, pero dentro de una misma familia emplearemos diferentes tamaños. Las Figuras 3.2 y 3.3 presentan una arquitectura representativa de cada una de las familias.

La familia **VGG** son arquitecturas basadas principalmente en CNN, funciones de activación *ReLU* y *max pooling*. Para los dos modelos seleccionados se puede destacar que la gran diferencia es que VGG19 cuenta con más capas convolucionales que VGG16. esto implica más parámetros y, por tanto, más memoria y tiempo de entrenamiento, pero es más hábil para resolver problemas más complejos.

Por otro lado, **EfficientNet** presentado por Google pretende mejorar la eficiencia del escalado de las redes convolucionales, optimizando aspectos como la anchura y la profundidad de las arquitecturas. Existe una familia completa de modelos preentrenados que van desde B0 a B7, donde el modelo más pequeño y compacto es B0. Por problemas de recursos se seleccionó B0, por su bajo número de parámetros y B3 con la finalidad de obtener un modelo más complejo que se espera que sea más capaz para resolver problemas más complejos.

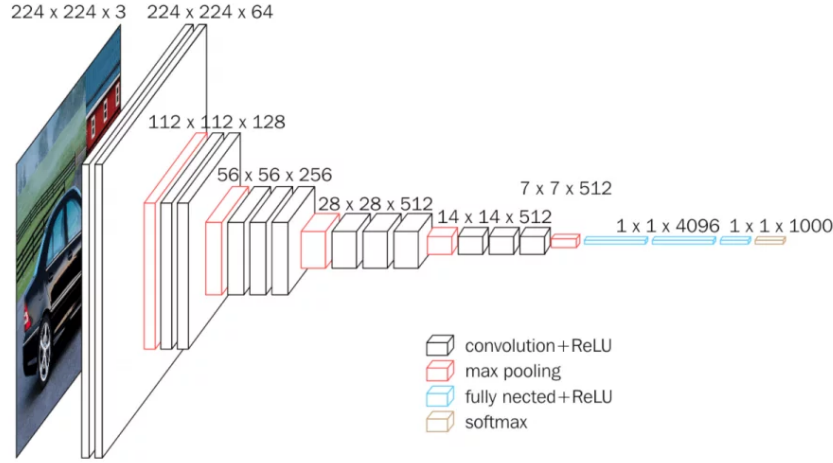


Figura 3.2: Arquitectura VGG16 [6]

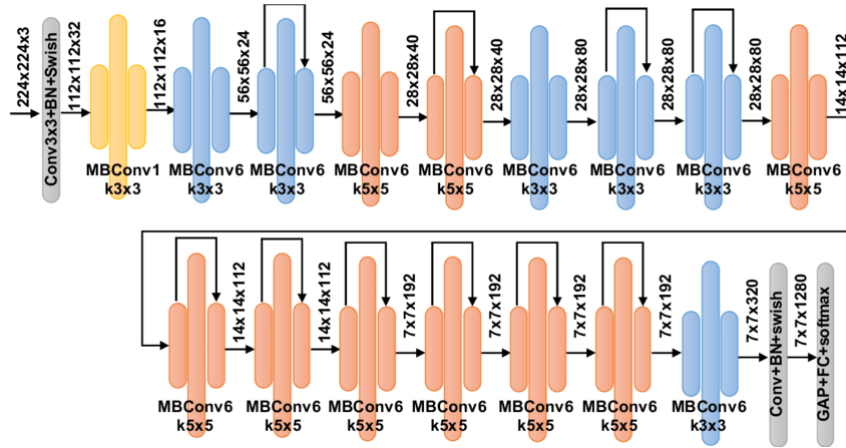


Figura 3.3: Arquitectura EfficientnetB0 [2]

Los modelos serán importados con la librería de **keras** de **Python**, serán modelos preentrenados en el dataset ImageNet, donde se importarán las capas convolucionales y las *Fully Connected* (FC) serán desechadas, para introducir nuevas que se ajusten al problema.

3.1.1. Construcción de los modelos

Dado que los modelos base se importan sin las capas de salida, estas deben definirse de forma explícita para ajustarse al número de clases del problema. Para ambas familias de modelos se sigue la misma estrategia de diseño, añadiendo las siguientes capas de manera secuencial:

1. **Global Average Pooling**, que calcula la media de los mapas de activación del último bloque convolucional y los transforma en un vector unidimensional. Esta opción se selecciona frente a *Flatten* con el objetivo de reducir significativamente el número de parámetros entrenables.
2. Una capa densa de 512 neuronas con función de activación *ReLU* y regularización L2, diseñada para mitigar la sobreespecialización durante el entrenamiento.
3. **Batch Normalization**, aplicada para normalizar la salida de la capa anterior y mejorar la estabilidad del proceso de entrenamiento.
4. **Dropout** con una tasa del 40
5. Una capa densa final cuyo número de neuronas depende del número de clases a predecir: 5 para el conjunto reducido y 15 para el conjunto completo. Como función de activación se emplea **softmax**, al permitir obtener una distribución de probabilidad sobre las clases de salida.

3.2. Estrategia de entrenamiento

Para el entrenamiento de los modelos se procede a aplicar la siguiente estrategia de entrenamiento donde se dividirá en dos etapas:

1. **Etapas 1 - Transfer learning**: se emplean pesos preentrenados y se congelan las capas convolucionales, entrenando únicamente las capas superiores, FC, añadidas para la tarea específica.
2. **Etapas 2 - Fine-tuning**: se entrenan tanto las capas superiores de la red preentrenada, permitiendo un ajuste más fino a los datos del problema.

En la primera etapa, debe seleccionarse un *learning rate* (tasa de aprendizaje) bajo, de tal manera que el aprendizaje de las FC sea progresivo. Cuando comience la segunda etapa, debe seleccionarse un *learning rate* aun menor, ya que se estarán ajustando las CNN, pero no deben perder las características aprendidas.

Otra estrategia que podría seguirse es ir descongelando las capas desde la salida hasta la cabeza del modelo de manera progresiva, pero no se ha contemplado en esta práctica.

3.2.1. *Callbacks*

Existen funciones que se pueden ejecutar durante el proceso de entrenamiento entre las que cabe destacar ***EarlyStop*** y ***LearningRateScheduler***.

EarlyStop permite detener el entrenamiento del modelo si no se satisface una condición durante un número determinado de épocas. Típicamente se selecciona la función de pérdidas sobre los datos de validación. Ante el problema que se procede a resolver se decide no emplear este método ya que al empezar la Etapa 2 y modificar todos los pesos se puede empeorar las métricas durante un número determinado de épocas y no interesa detener el entrenamiento.

LearningRateScheduler se encarga de modificar el *learning rate* a medida que se ejecutan las diferentes épocas de entrenamiento. Existen diferentes estrategias, pero se empleó en esta práctica el decrementar el *learning rate* en un 10% cada 5 épocas.

3.3. Configuración del entrenamiento

3.3.1. Optimizadores

Durante el entrenamiento, se considerarán dos optimizadores ampliamente utilizados en redes neuronales profundas:

- **SGD** (*Stochastic Gradient Descent*), que ofrece una convergencia más estable y controlada, especialmente en escenarios donde se requiere un ajuste progresivo del aprendizaje [7].
- **Adam**, que adapta dinámicamente la tasa de aprendizaje para cada parámetro, permitiendo una convergencia más rápida en las primeras fases del entrenamiento [5].

3.3.2. Función de pérdida

Para la función de pérdida se contemplan dos alternativas habituales en problemas de clasificación multiclase:

- `categorical_crossentropy`, utilizada cuando las etiquetas objetivo están codificadas mediante *one-hot encoding*.
- `sparse_categorical_crossentropy`, empleada cuando las etiquetas se representan como un vector de enteros.

Dado que en este trabajo las etiquetas se encuentran codificadas como valores enteros, se empleará `sparse_categorical_crossentropy` como función de pérdida principal.

A pesar de haberse aplicado un filtrado previo al conjunto de datos, resulta imprescindible realizar un preprocesado adicional antes de introducir las imágenes en los modelos de aprendizaje profundo, con el fin de adaptarlas a los requisitos específicos de cada arquitectura. Tanto las redes de la familia **VGG** como **EfficientNet** requieren imágenes de entrada con una resolución fija de 224×224 píxeles. En consecuencia, todas las imágenes son redimensionadas a este tamaño de forma consistente.

Adicionalmente, cada una de estas familias de modelos incorpora un esquema de preprocesado propio, diseñado para normalizar los valores de entrada y facilitar la convergencia durante el entrenamiento. Este preprocesado incluye, entre otros aspectos, la estandarización de los canales de color y la adecuación de las intensidades de los píxeles a la distribución esperada por los modelos preentrenados.

Dado que el conjunto de datos disponible es limitado, se emplean técnicas de *data augmentation* como estrategia para mejorar la capacidad de generalización de los modelos y reducir el riesgo de *overfitting*. Estas técnicas permiten generar variaciones artificiales de las imágenes de entrenamiento manteniendo su etiqueta original, incrementando así la diversidad efectiva del conjunto de datos.

En este trabajo se han seleccionado dos transformaciones de *data augmentation* de bajo impacto semántico: la inversión horizontal aleatoria (*Random Horizontal Flip*), que simula imágenes especulares, y la rotación aleatoria (*Random Rotation*), aplicada con un ángulo reducido para preservar la coherencia visual del contenido. Ambas transformaciones se aplican exclusivamente al conjunto de entrenamiento, garantizando que los conjuntos de validación y prueba permanezcan inalterados.

3.4. Métricas de evaluación

Para evaluar el rendimiento de los modelos entrenados se utilizarán tanto métricas durante el entrenamiento como métricas finales sobre el conjunto de test. En particular, se analizarán:

- La evolución de la **función de pérdida** y la **accuracy** durante el entrenamiento y la validación.
- Las métricas clásicas de clasificación: **accuracy**, **precision**, **recall** y **F1-score**.
- La **matriz de confusión**, que permitirá analizar el comportamiento del modelo por clase y detectar posibles confusiones sistemáticas.
- **Top- k accuracy**: esta métrica se emplea habitualmente en problemas de clasificación con un número elevado de clases. Dado que la capa de salida del modelo utiliza la función de activación *softmax*, se obtiene una distribución de probabilidad sobre todas las clases posibles. La métrica *Top- k accuracy* considera una predicción correcta si la etiqueta real se encuentra entre las k clases con mayor probabilidad asignada por el modelo. En este trabajo, para el conjunto de datos reducido se evalúa esta métrica con $k = 3$, mientras que para el conjunto de datos completo se analizan los casos $k = 3$ y $k = 5$.

Estas métricas proporcionan una visión global y detallada del rendimiento de los modelos, permitiendo una comparación objetiva entre arquitecturas, estrategias de entrenamiento y conjuntos de datos.

Capítulo 4

Resultados del entrenamiento

En este capítulo se presentan y analizan los resultados obtenidos durante el proceso de entrenamiento de los distintos modelos propuestos, siguiendo la metodología descrita en el Capítulo 3. El objetivo principal es evaluar de forma sistemática el impacto de la calidad de los datos, la complejidad del problema y las decisiones de diseño del modelo sobre el rendimiento final.

Tal y como se ha descrito previamente, el estudio experimental se estructura en tres escenarios diferenciados, definidos en función del preprocesado aplicado al conjunto de datos y del número de clases consideradas. Cada uno de estos escenarios plantea un nivel de dificultad distinto.

El **primer escenario** corresponde al conjunto de datos reducido y filtrado, en el que se consideran únicamente cinco clases y se ha aplicado un proceso de filtrado para homogeneizar el tamaño y la resolución de las imágenes. Las cinco clases seleccionadas son ‘*calling*’, ‘*clapping*’, ‘*cycling*’, ‘*dancing*’, ‘*drinking*’. Este escenario representa un problema de menor complejidad y mayor control sobre la calidad de los datos, por lo que se espera que los modelos alcancen su mejor rendimiento. Por este motivo, constituye la mayor experimentación de configuraciones de entrenamiento, comparando distintas arquitecturas, tamaños de modelo, optimizadores y estrategias de entrenamiento.

El **segundo escenario** utiliza el conjunto de datos completo filtrado, manteniendo las quince clases originales. En este caso, el objetivo es extraer conclusiones al aumentar la complejidad del problema y analizar la capacidad de generalización de los modelos, evaluando específicamente el impacto de incrementar el número de

clases. En este escenario se evalúan únicamente las dos configuraciones de entrenamiento que mostraron un mejor rendimiento en el primer escenario.

Finalmente, el **tercer escenario** se basa en un conjunto de datos reducido sin aplicar el filtrado. Este escenario permite analizar de forma aislada el efecto del preprocesado y del filtrado de imágenes, comparando el impacto de haber considerado la información de ancho, alto y resolución. En este caso, se evalúa únicamente la configuración de entrenamiento que obtuvo los mejores resultados en el primer escenario.

En las secciones siguientes se presentan los resultados correspondientes a cada escenario. Para cada experimento se analizan aspectos como el número de parámetros entrenables, la evolución de la función de pérdida y de las métricas de evaluación durante el entrenamiento, así como el rendimiento final sobre el conjunto de test mediante métricas clásicas de clasificación y matrices de confusión. Este análisis permite extraer conclusiones fundamentadas sobre la idoneidad de las distintas configuraciones y estrategias de entrenamiento evaluadas.

Para las matrices de confusión, se utilizarán índices numéricos que hacen referencia directa a las etiquetas del conjunto de datos. La Tabla 4.1 presenta la correspondencia entre cada índice y su etiqueta asociada, y será la referencia empleada a lo largo del análisis.

Tabla 4.1: Asignación de índices (0–14) a etiquetas de clase.

Índice	Etiqueta	Índice	Etiqueta	Índice	Etiqueta
0	calling	5	eating	10	running
1	clapping	6	fighting	11	sitting
2	cycling	7	hugging	12	sleeping
3	dancing	8	laughing	13	texting
4	drinking	9	listening_to_music	14	using_laptop

4.1. Escenario 1: Entrenamiento sobre el dataset reducido y filtrado

Este escenario pretende buscar una buena configuración para resolver el problema de *Human Action Recognition* seleccionando las primeras cinco clases y filtrando aquellas imágenes que satisfacen unos criterios de ancho, alto y resolución. Para

buscar dicha configuración se han realizado varios experimentos donde se destacarán los seis siguientes experimentos.

4.1.1. Experimento 1: VGG16 con Adam y Batch Size 8

En este experimento se emplea la arquitectura **VGG16** junto con el optimizador **Adam**, utilizando un *batch size* = 8. El objetivo principal es establecer un primer punto de referencia sobre el dataset reducido y filtrado, evaluando el comportamiento del modelo bajo una configuración clásica de *transfer learning* seguida de *fine-tuning*.

Configuración del entrenamiento

La configuración empleada en este experimento se resume en la Tabla 4.2.

Tabla 4.2: Configuración del Experimento 1 del Escenario 1

Modelo	VGG16
Optimizador	Adam
Batch size	8
Learning Rate (Transfer Learning)	1×10^{-4}
Épocas (Transfer Learning)	10
Learning Rate (Fine-tuning)	1×10^{-5}
Épocas (Fine-tuning)	10
Número total de parámetros	14.981.957

Evolución del entrenamiento

La Figura 4.1 muestra la evolución de la *accuracy*, la función de pérdida, la tasa de aprendizaje y el número de parámetros entrenables a lo largo de las fases de *transfer learning* y *fine-tuning*.

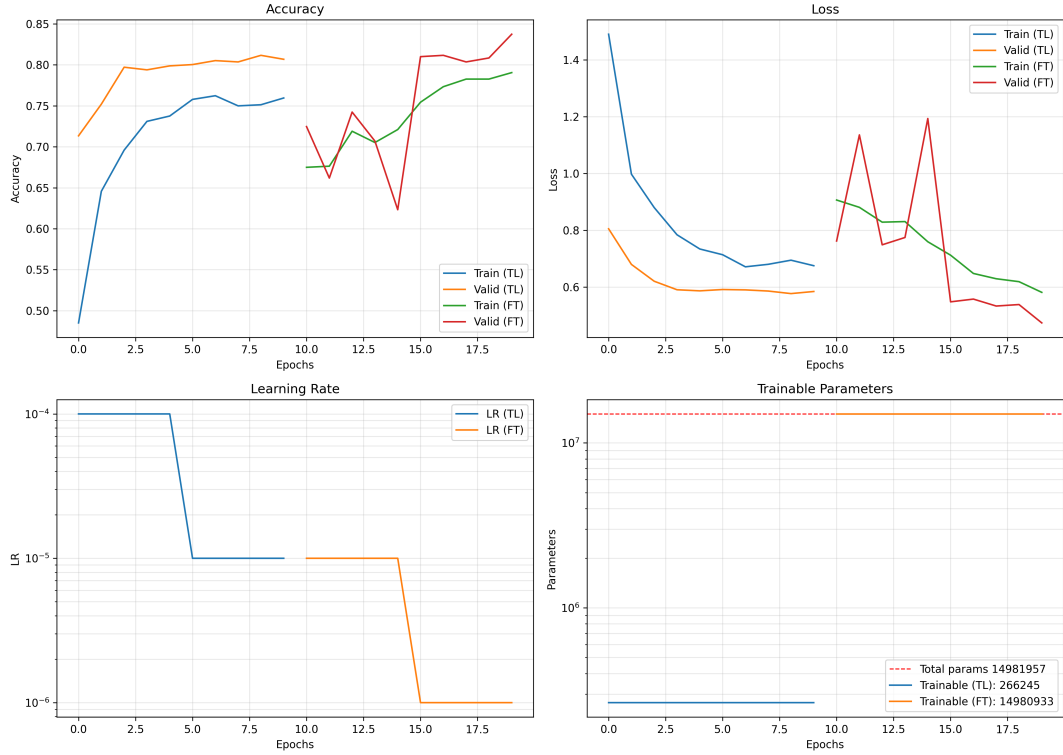


Figura 4.1: Evolución del entrenamiento durante las fases de *transfer learning* y *fine-tuning* — Escenario 1, Experimento 1

Durante la fase de *transfer learning* se observa una convergencia progresiva y estable tanto en entrenamiento como en validación. La curva de *accuracy* de entrenamiento se mantiene ligeramente por debajo de la de validación, lo cual indica ligeros síntomas de *underfitting*. Este comportamiento puede atribuirse a la aplicación de técnicas de regularización como *data augmentation*, *dropout* y regularización L2, que introducen ruido controlado durante el entrenamiento y dificultan el ajuste exacto sobre los datos de entrenamiento. Este efecto resulta beneficioso para mejorar la capacidad de generalización del modelo.

Al inicio de la fase de *fine-tuning* se aprecia una caída puntual en la *accuracy* y un incremento temporal de la función de pérdida. Este comportamiento es esperable, ya que en esta etapa se comienzan a ajustar simultáneamente todos los pesos de las capas convolucionales previamente congeladas, obligando al modelo a reajustar las representaciones aprendidas. Tras este periodo de adaptación, el modelo recupera una tendencia positiva, alcanzando valores superiores de *accuracy* y una reducción sostenida de la pérdida. Con la evolución observada podría haberse dejado entrenar unas épocas más el modelo.

Análisis de la matriz de confusión

La matriz de confusión obtenida sobre el conjunto de test se muestra en la Figura 4.2.

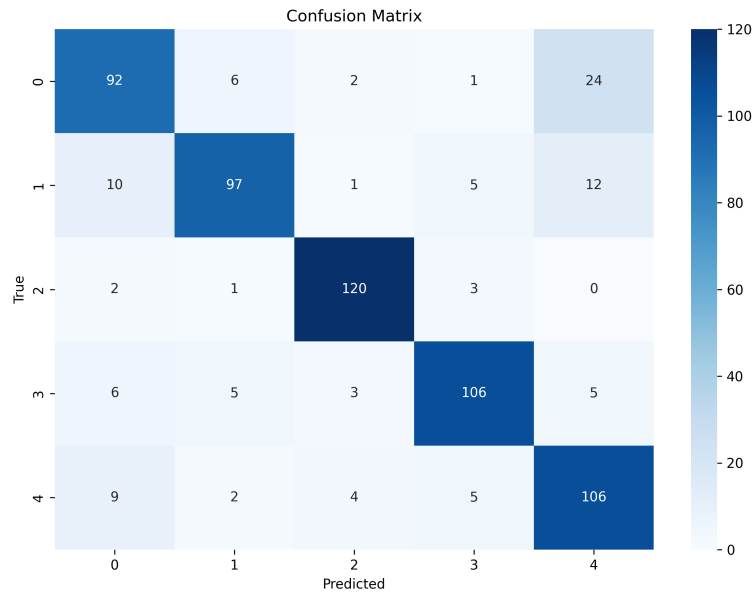


Figura 4.2: Matriz de confusión en el conjunto de test — Escenario 1, Experimento 1

La matriz de confusión refleja un buen rendimiento global del modelo en todas las clases consideradas. La mayoría de las muestras se clasifican correctamente, observándose confusiones puntuales entre determinadas clases. No se detecta un sesgo claro hacia ninguna clase concreta ni patrones sistemáticos de error.

Resultados finales

Los resultados cuantitativos finales del experimento se recogen en la Tabla 4.3.

Tabla 4.3: Resultados finales del Escenario 1, Experimento 1

Métrica	Train	Validación	Test
Accuracy	0.83	0.84	0.82
Loss	0.49	0.41	—
Precision (macro)	—	—	0.83
Recall (macro)	—	—	0.81
F1-score (macro)	—	—	0.82
Top-3 Accuracy	—	—	0.96

En conjunto, los resultados obtenidos indican que esta configuración ofrece un buen equilibrio entre la estabilidad del entrenamiento y la capacidad de generalización, constituyendo una referencia sólida para comparar configuraciones posteriores dentro de este escenario.

4.1.2. Experimento 2: VGG16 con SGD y Batch Size 8

En este experimento se mantiene la arquitectura **VGG16** y el *batch size*, sustituyendo el optimizador Adam por **SGD**. El objetivo es analizar el impacto de un optimizador con una convergencia más lenta y progresiva sobre el mismo conjunto de datos reducido y filtrado.

Configuración del entrenamiento

La configuración del experimento se resume en la Tabla 4.4. A diferencia del experimento anterior, se incrementa el número de épocas tanto en la fase de *transfer learning* como en la de *fine-tuning*, alcanzando un total de 20 épocas en cada etapa, con el fin de permitir una convergencia más gradual característica del optimizador SGD.

Tabla 4.4: Configuración del Experimento 2 del Escenario 1

Modelo	VGG16
Optimizador	SGD
Batch size	8
Learning Rate (Transfer Learning)	1×10^{-4}
Épocas (Transfer Learning)	20
Learning Rate (Fine-tuning)	1×10^{-5}
Épocas (Fine-tuning)	20
Número total de parámetros	14.981.957

Evolución del entrenamiento

La Figura 4.3 muestra la evolución de las métricas de entrenamiento y validación, así como la programación de la tasa de aprendizaje y el número de parámetros entrenables.

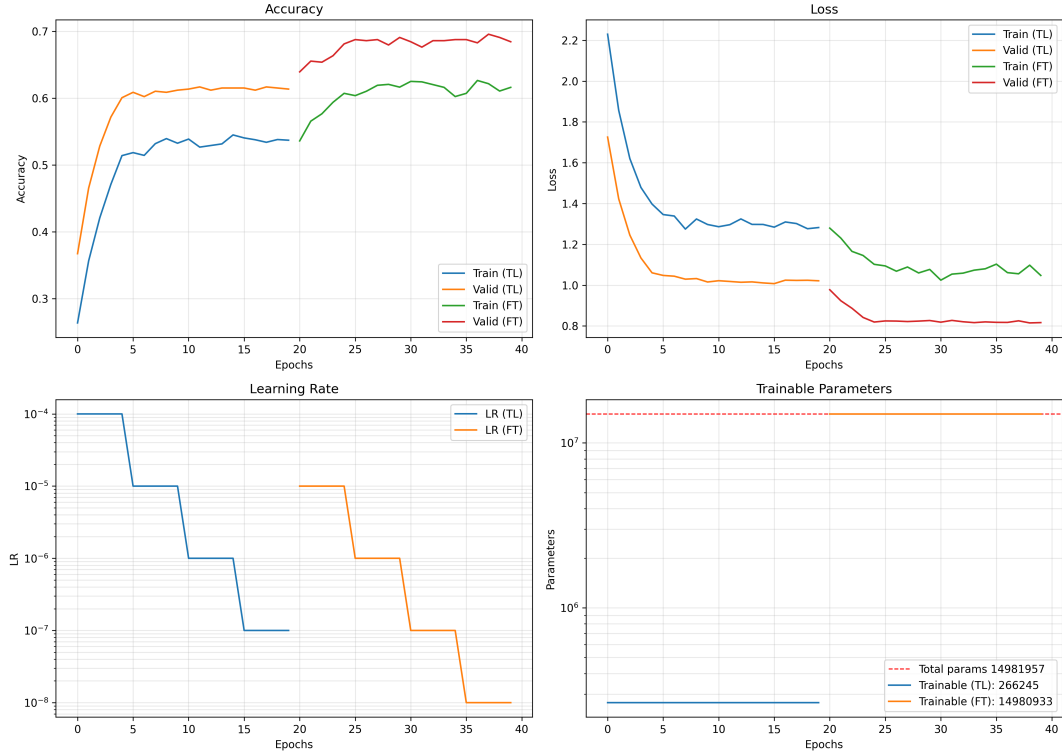


Figura 4.3: Evolución del entrenamiento durante las fases de *transfer learning* y *fine-tuning* — Escenario 1, Experimento 2

Durante la fase de *transfer learning* se observa una evolución claramente más lenta y progresiva en comparación con el Experimento 1, lo cual es coherente con el comportamiento esperado del optimizador SGD. A pesar del aumento en el número de épocas, la *accuracy* de entrenamiento permanece significativamente por debajo de la de validación, indicando un claro fenómeno de *underfitting*.

La aplicación del *learning rate scheduler* se aprecia de forma clara en la evolución de la tasa de aprendizaje. Al realizar la transición de *transfer learning* a *fine-tuning*, se incrementa el valor del *learning rate*, lo que permite al modelo escapar parcialmente de un estado de estancamiento observado al final de la primera etapa. No obstante, aunque esta estrategia introduce una mejora inicial, el modelo no logra alcanzar niveles de rendimiento comparables a los obtenidos con Adam.

Análisis de la matriz de confusión

La matriz de confusión obtenida sobre el conjunto de test se muestra en la Figura 4.4.

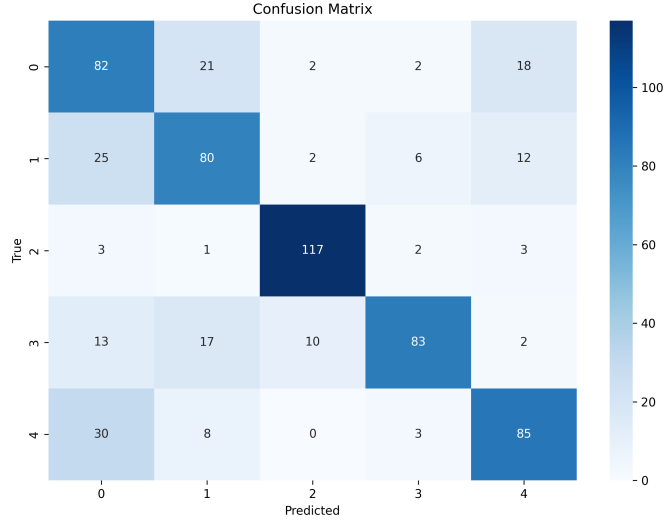


Figura 4.4: Matriz de confusión en el conjunto de test — Escenario 1, Experimento 2

La matriz de confusión refleja un mayor número de errores de clasificación en comparación con el Experimento 1, observándose confusiones más frecuentes entre varias clases.

Resultados finales

Los resultados cuantitativos finales del experimento se recogen en la Tabla 4.5.

Tabla 4.5: Resultados finales del Escenario 1, Experimento 2

Métrica	Train	Validación	Test
Accuracy	0.60	0.70	0.70
Loss	1.11	0.85	—
Precision (macro)	—	—	0.69
Recall (macro)	—	—	0.70
F1-score (macro)	—	—	0.69
Top-3 Accuracy	—	—	0.96

En conjunto, los resultados obtenidos indican que, bajo las condiciones evaluadas, el uso de SGD conduce a un entrenamiento más lento y a un marcado *underfitting*, incluso al aumentar el número de épocas y aplicar estrategias de ajuste del *learning rate*. En comparación con el Experimento 1, esta configuración resulta claramente inferior.

4.1.3. Experimento 3: VGG16 con Adam y Batch Size 64

En este experimento se parte de la configuración que mejores resultados obtuvo en experimentos previos, es decir, el Experimento 1, manteniendo la arquitectura **VGG16**, el optimizador **Adam** y los mismos hiperparámetros de aprendizaje. La única modificación introducida consiste en aumentar el tamaño *Batch Size* de 8 a 64, con el objetivo de analizar la capacidad de generalización del modelo.

Configuración del entrenamiento

La configuración empleada en este experimento se resume en la Tabla 4.6. Todos los parámetros se mantienen constantes respecto al Experimento 1, salvo el tamaño del lote.

Tabla 4.6: Configuración del Experimento 3 del Escenario 1

Modelo	VGG16
Optimizador	Adam
Batch size	64
Learning Rate (Transfer Learning)	1×10^{-4}
Épocas (Transfer Learning)	10
Learning Rate (Fine-tuning)	1×10^{-5}
Épocas (Fine-tuning)	10
Número total de parámetros	14.981.957

Evolución del entrenamiento

La Figura 4.5 muestra la evolución de las métricas de entrenamiento y validación, así como la programación de la tasa de aprendizaje y el número de parámetros entrenables.

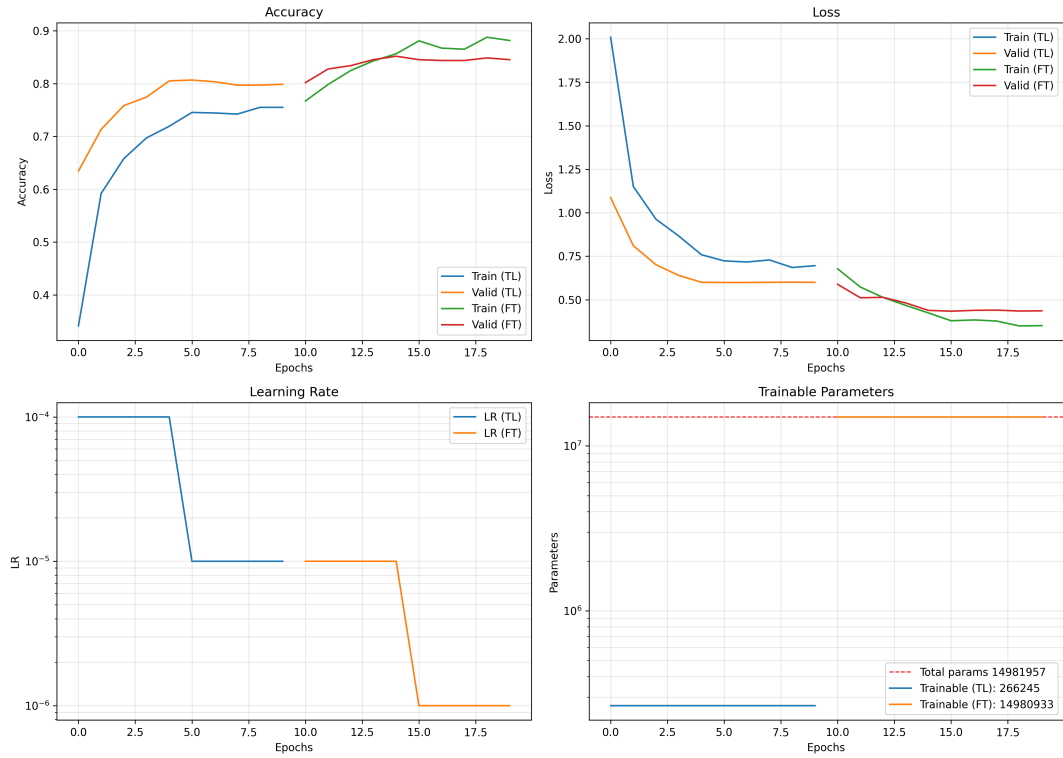


Figura 4.5: Evolución del entrenamiento durante las fases de *transfer learning* y *fine-tuning* — Escenario 1, Experimento 3

En comparación con el Experimento 1, el uso de un *batch size* mayor da lugar a una evolución más estable de las curvas de *accuracy* y pérdida, lidiando con una mejor generalización y haciendo frente a los problemas de underfitting presentado, sobre todo, esto se soluciona al iniciar la etapa de *finetuning*.

Análisis de la matriz de confusión

La matriz de confusión obtenida sobre el conjunto de test se muestra en la Figura 4.6.

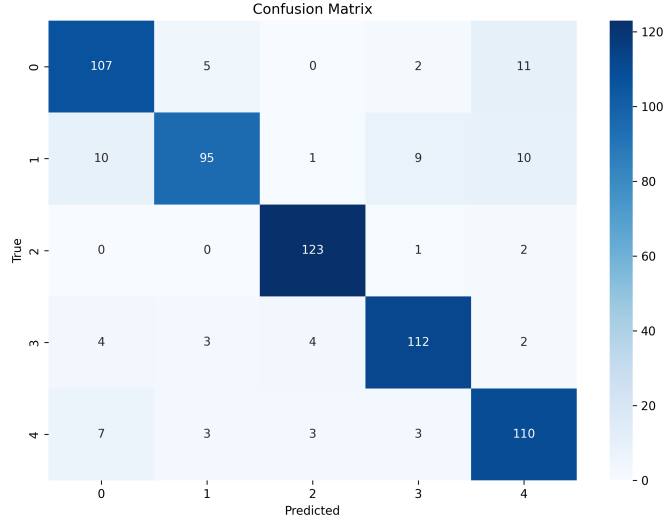


Figura 4.6: Matriz de confusión en el conjunto de test — Escenario 1, Experimento 3

La matriz de confusión refleja una mejora considerable en comparación con los experimentos previos. Esta mejora también se observará a continuación en las métricas finales de evaluación.

Resultados finales

Los resultados cuantitativos finales del experimento se recogen en la Tabla 4.7.

Tabla 4.7: Resultados finales del Escenario 1, Experimento 3

Métrica	Train	Validación	Test
Accuracy	0.88	0.85	0.87
Loss	0.35	0.44	—
Precision (macro)	—	—	0.87
Recall (macro)	—	—	0.87
F1-score (macro)	—	—	0.87
Top-3 Accuracy	—	—	0.99

En conjunto, los resultados muestran una mejora clara en la estabilidad del entrenamiento y en la capacidad de generalización respecto al Experimento 1, confirmando que el aumento del *batch size* contribuye a mitigar los efectos de regularización excesiva y a facilitar una transición más suave entre las fases de *transfer learning* y *fine-tuning*. Esta configuración se posiciona como una de las más sólidas dentro del Escenario 1.

4.1.4. Experimento 4: EfficientNetB0 con Adam y Batch Size 64

En este experimento se introduce la arquitectura **EfficientNetB0**, manteniendo el optimizador **Adam** y el *batch size* = 64, que mostró un comportamiento más estable. El objetivo es evaluar si esta arquitectura permite mejorar el compromiso entre el rendimiento y la complejidad del modelo sobre el dataset reducido y filtrado.

Configuración del entrenamiento

La configuración empleada en este experimento se resume en la Tabla 4.8. En comparación con VGG16, EfficientNetB0 presenta un número significativamente menor de parámetros totales, lo que se espera que favorezca la eficiencia computacional sin penalizar el rendimiento.

Tabla 4.8: Configuración del Experimento 4 del Escenario 1

Modelo	EfficientNetB0
Optimizador	Adam
Batch size	64
Learning Rate (Transfer Learning)	1×10^{-4}
Épocas (Transfer Learning)	10
Learning Rate (Fine-tuning)	1×10^{-5}
Épocas (Fine-tuning)	10
Número total de parámetros	4.710.056

Evolución del entrenamiento

La Figura 4.7 muestra la evolución de la *accuracy*, la función de pérdida, la tasa de aprendizaje y el número de parámetros entrenables durante las fases de *transfer learning* y *fine-tuning*.

A lo largo del entrenamiento se observa un comportamiento estable y consistente tanto en entrenamiento como en validación, sin indicios claros de *overfitting* o *underfitting*. La transición entre las fases de *transfer learning* y *fine-tuning* se realiza de forma suave, sin empeorar la métrica de *accuracy*, lo que indica una correcta adaptación de los pesos convolucionales al dominio del problema.

Este comportamiento contrasta positivamente con el observado en configuraciones basadas en VGG16, donde la transición a *fine-tuning* mostraba mayor inestabilidad.

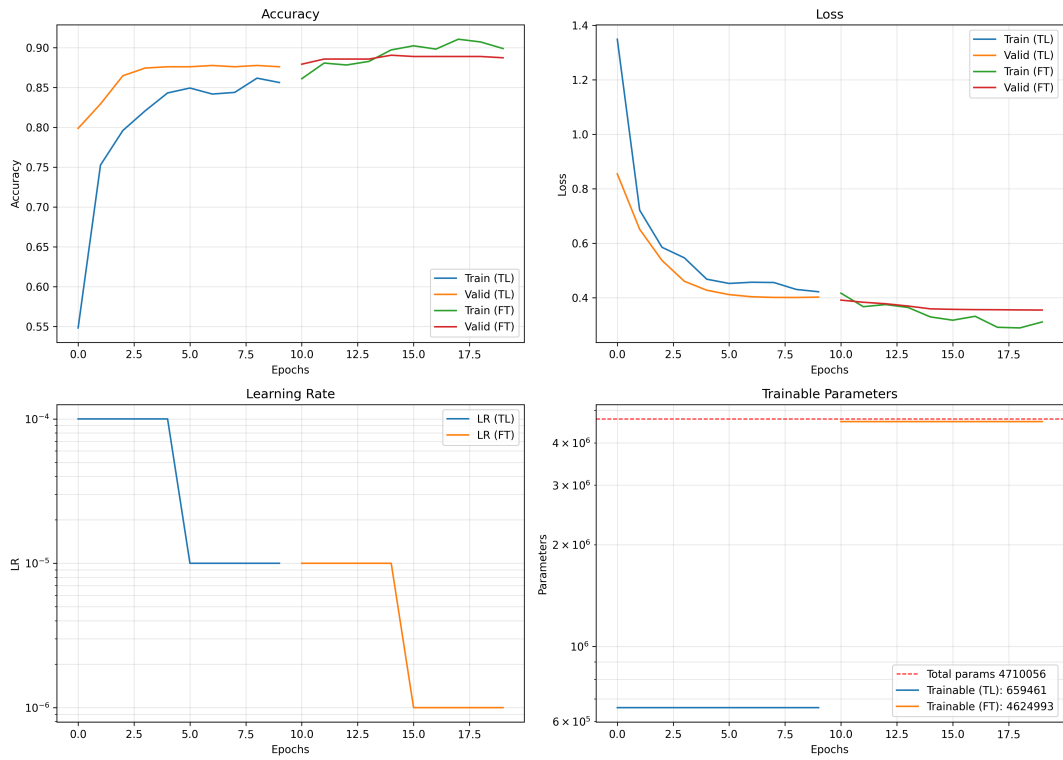


Figura 4.7: Evolución del entrenamiento durante las fases de *transfer learning* y *fine-tuning* — Escenario 1, Experimento 4

La arquitectura EfficientNetB0 no solo lo hace mejor si no que necesita muchos menos parámetros de entrenamiento.

Análisis de la matriz de confusión

La matriz de confusión obtenida sobre el conjunto de test se muestra en la Figura 4.8.

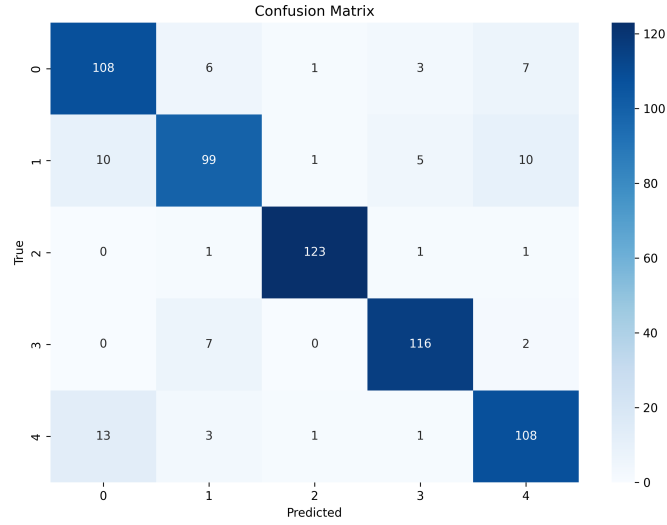


Figura 4.8: Matriz de confusión en el conjunto de test — Escenario 1, Experimento 4

La matriz de confusión refleja un alto grado de acierto en todas las clases, con confusiones puntuales y no sistemáticas entre algunas acciones. No se observa un sesgo dominante ni una degradación del rendimiento en clases concretas, lo que confirma un aprendizaje equilibrado de las distintas categorías.

Resultados finales

Los resultados cuantitativos finales del experimento se recogen en la Tabla 4.9.

Tabla 4.9: Resultados finales del Escenario 1, Experimento 4

Métrica	Train	Validación	Test
Accuracy	0.90	0.89	0.88
Loss	0.31	0.35	0.88
Precision (macro)	—	—	0.88
Recall (macro)	—	—	0.88
F1-score (macro)	—	—	0.88
Top-3 Accuracy	—	—	0.99

En conjunto, los resultados muestran que EfficientNetB0 ofrece un excelente compromiso entre rendimiento, estabilidad del entrenamiento y eficiencia en el número de parámetros. Esta configuración iguala o mejora el rendimiento obtenido con VGG16.

4.1.5. Experimento 5: EfficientNetB3 con Adam y Batch Size 64

En este experimento se evalúa la arquitectura **EfficientNetB3**, manteniendo la misma configuración empleada en el Experimento 4 (optimizador Adam, *batch size* = 64 y mismos hiperparámetros de aprendizaje). El objetivo es analizar el impacto de incrementar la complejidad del modelo y el número de parámetros entrenables sobre el rendimiento, comparándolo directamente con EfficientNetB0.

Configuración del entrenamiento

La configuración del experimento se resume en la Tabla 4.10. La principal diferencia respecto al experimento anterior es el aumento significativo en el número de parámetros del modelo, asociado a una arquitectura más profunda y ancha.

Tabla 4.10: Configuración del Experimento 5 del Escenario 1

Modelo	EfficientNetB3
Optimizador	Adam
Batch size	64
Learning Rate (Transfer Learning)	1×10^{-4}
Épocas (Transfer Learning)	10
Learning Rate (Fine-tuning)	1×10^{-5}
Épocas (Fine-tuning)	10
Número total de parámetros	11.399.469

Evolución del entrenamiento

La Figura 4.9 muestra la evolución de las métricas de entrenamiento y validación, así como la tasa de aprendizaje y el número de parámetros entrenables.

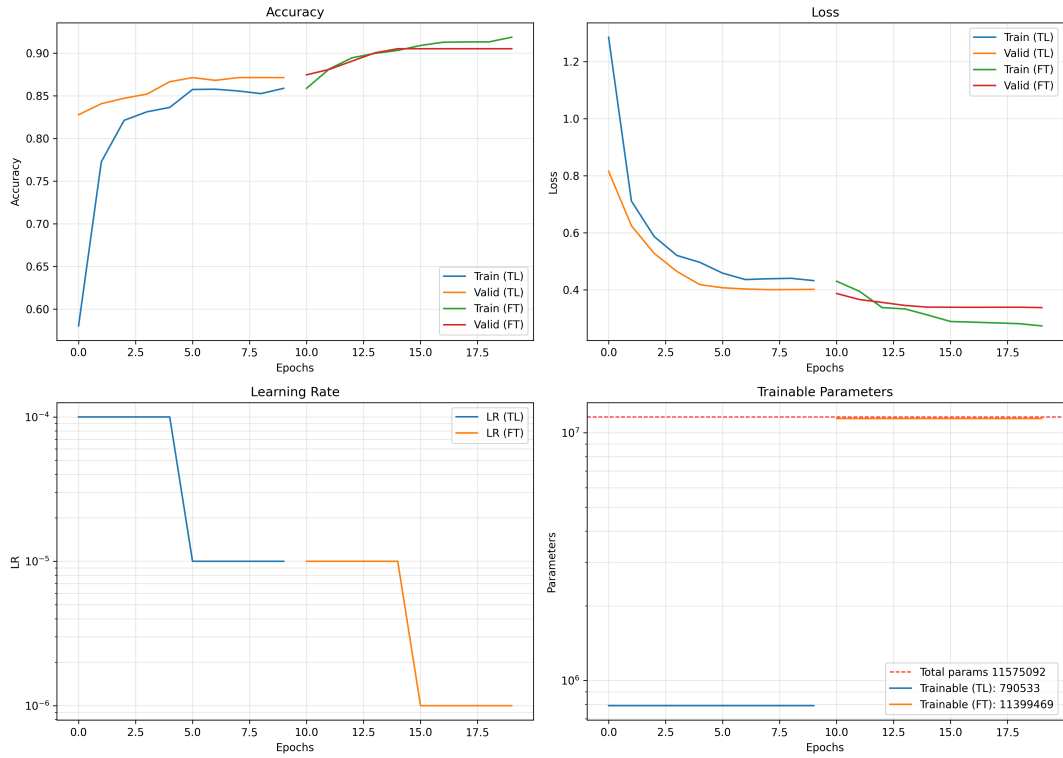


Figura 4.9: Evolución del entrenamiento durante las fases de *transfer learning* y *fine-tuning* — Escenario 1, Experimento 5

El entrenamiento presenta un comportamiento estable y una convergencia adecuada tanto en entrenamiento como en validación, sin evidencias claras de *overfitting*. Sin embargo, en comparación con EfficientNetB0, la mejora observada en la *accuracy* y en la función de pérdida es mínima, a pesar del incremento notablemente las capacidades del modelo.

Análisis de la matriz de confusión

La matriz de confusión obtenida sobre el conjunto de test se muestra en la Figura 4.10.

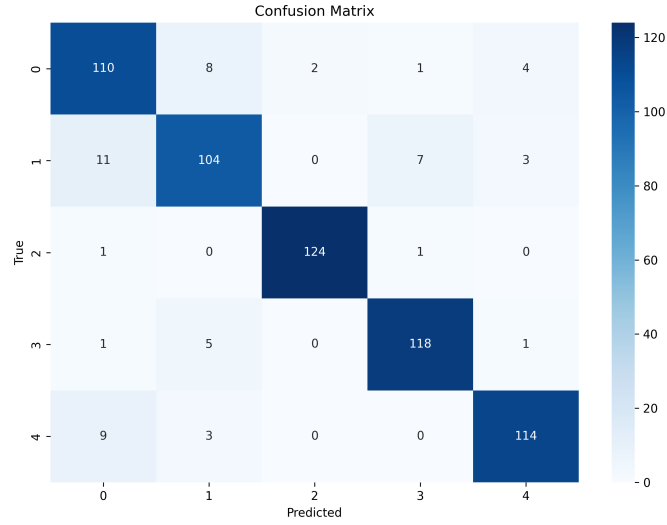


Figura 4.10: Matriz de confusión en el conjunto de test — Escenario 1, Experimento 5

La matriz de confusión refleja un alto grado de acierto en todas las clases, con confusiones puntuales y no sistemáticas.

Resultados finales

Los resultados cuantitativos finales del experimento se recogen en la Tabla 4.11.

Tabla 4.11: Resultados finales del Escenario 1, Experimento 5

Métrica	Train	Validación	Test
Accuracy	0.92	0.90	0.91
Loss	0.27	0.34	—
Precision (macro)	—	—	0.91
Recall (macro)	—	—	0.91
F1-score (macro)	—	—	0.91
Top-3 Accuracy	—	—	0.99

En conjunto, los resultados indican que el aumento de complejidad introducido por EfficientNetB3 mejora sutilmente el rendimiento respecto a EfficientNetB0, pero no compensa debido al incremento en el número de parámetros entrenables y el coste computacional asociado. Por este motivo, EfficientNetB0 se considera una opción más eficiente y equilibrada para este problema dentro del Escenario 1.

4.1.6. Experimento 6: VGG19 con Adam y Batch Size 64

En este experimento se evalúa la arquitectura **VGG19**, manteniendo la misma configuración empleada en el Experimento 3 (optimizador Adam, *batch size* = 64 y mismos hiperparámetros de aprendizaje). El objetivo es analizar si el aumento de profundidad de la arquitectura VGG, y por tanto del número de parámetros, conduce a una mejora del rendimiento frente a VGG16 en el mismo escenario experimental.

Configuración del entrenamiento

La configuración del experimento se resume en la Tabla 4.12. La única diferencia respecto al Experimento 3 es el uso de la arquitectura VGG19, que incorpora un mayor número de capas convolucionales y, en consecuencia, un incremento significativo en el número total de parámetros del modelo.

Modelo	VGG19
Optimizador	Adam
Batch size	64
LR (Transfer Learning)	1×10^{-4}
Épocas (Transfer Learning)	10
LR (Fine-tuning)	1×10^{-5}
Épocas (Fine-tuning)	10

Tabla 4.12: Configuración del Experimento 6 del Escenario 1

Evolución del entrenamiento

La Figura 4.11 muestra la evolución de las métricas de entrenamiento y validación durante las fases de *transfer learning* y *fine-tuning*.

El entrenamiento presenta una convergencia estable y sin oscilaciones abruptas, con curvas de *accuracy* y pérdida bien alineadas entre entrenamiento y validación. No obstante, al comparar estas curvas con las obtenidas en el Experimento 3, no se observa una mejora sustancial en la velocidad de convergencia ni en la estabilidad del proceso de entrenamiento que justifique el incremento de complejidad introducido por VGG19.

Durante la fase de *fine-tuning*, el modelo logra ajustar correctamente los pesos convolucionales, pero la ganancia en *accuracy* es similar, e incluso ligeramente inferior, a la obtenida con VGG16 bajo la misma configuración.

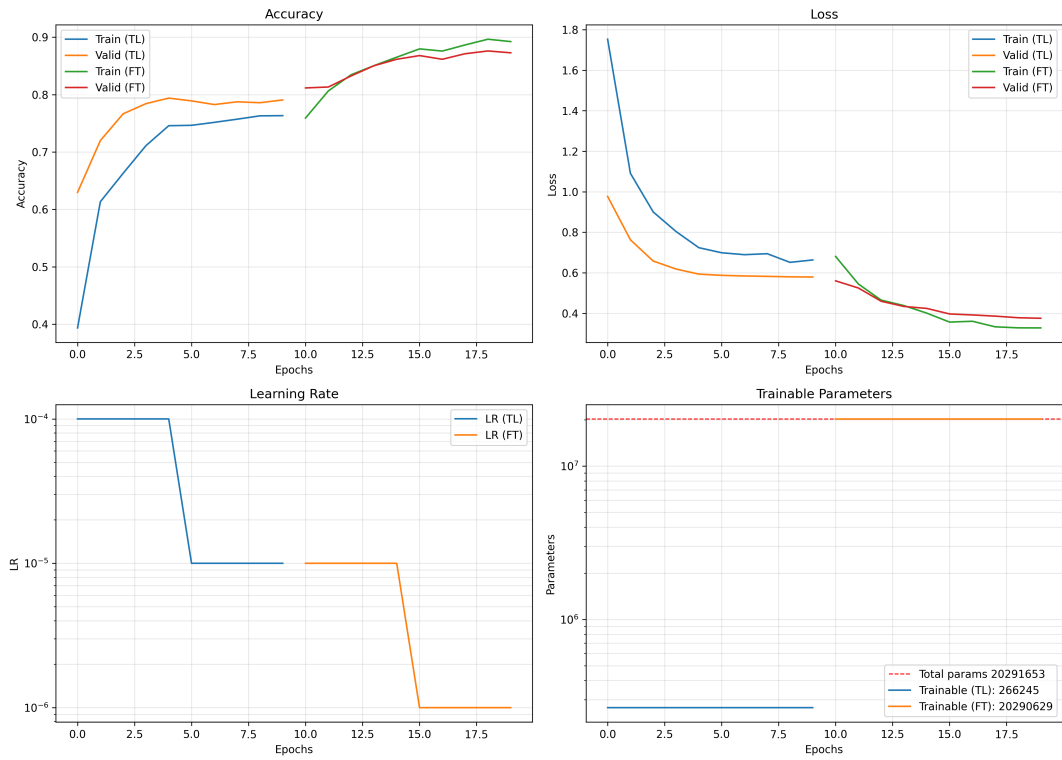


Figura 4.11: Evolución del entrenamiento durante las fases de *transfer learning* y *fine-tuning* — Escenario 1, Experimento 6

Análisis de la matriz de confusión

La matriz de confusión obtenida sobre el conjunto de test se muestra en la Figura 4.12.

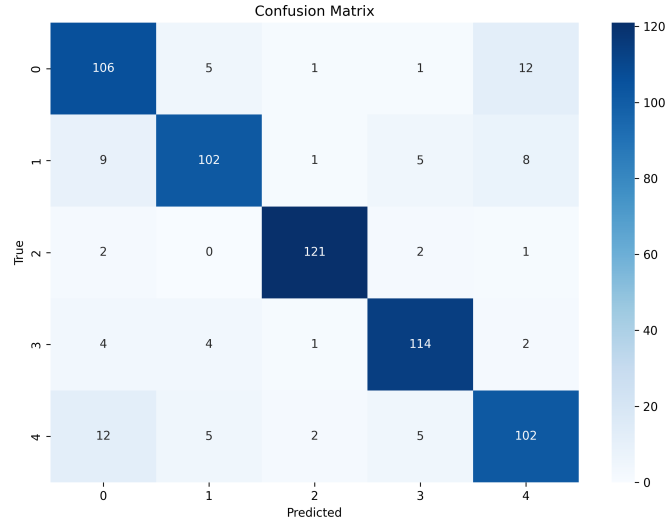


Figura 4.12: Matriz de confusión en el conjunto de test — Escenario 1, Experimento 6

La matriz de confusión refleja un alto grado de acierto en todas las clases, sin claros sesgos.

Resultados finales

Los resultados cuantitativos finales del experimento se recogen en la Tabla 4.13.

Métrica	Train	Validación	Test
Accuracy	0.89	0.87	0.87
Loss	0.33	0.38	—
Precision (macro)	—	—	0.87
Recall (macro)	—	—	0.87
F1-score (macro)	—	—	0.87
Top-3 Accuracy	—	—	0.99

Tabla 4.13: Resultados finales del Escenario 1, Experimento 6

En comparación con el Experimento 3, el uso de VGG19 no aporta una mejora significativa en términos de rendimiento sobre el conjunto de test, a pesar del aumento en el número de parámetros entrenables y del coste computacional asociado. Por tanto, VGG19 queda descartado como candidato frente a configuraciones más eficientes evaluadas previamente.

4.2. Escenario 2: Entrenamiento sobre el dataset filtrado con 15 clases

Tras identificar en el Escenario 1 una configuración robusta y bien balanceada para el problema de *Human Action Recognition* sobre un conjunto reducido de clases, en este segundo escenario se incrementa la complejidad del problema manteniendo constante el filtrado de los datos. En concreto, se emplea el **dataset filtrado completo**, considerando ahora las **15 clases originales** del conjunto de datos.

El objetivo de este escenario es analizar cómo se comportan las arquitecturas seleccionadas cuando se enfrentan a un problema más exigente desde el punto de vista semántico, evaluando su capacidad de generalización al aumentar el número de clases, pero sin introducir ruido adicional derivado de imágenes con resoluciones muy dispares. De este modo, el filtrado por ancho, alto y resolución se mantiene como en el escenario anterior.

A diferencia del Escenario 1, donde se realizó una exploración más amplia del espacio de configuraciones, en este caso el estudio experimental se simplifica. Únicamente se evalúan las arquitecturas **VGG16**, y *EfficientNetB3* bajo una misma configuración. No se decide probar con *EfficientNetB0* ya que se presupone que para estos datos el modelo es muy complejo y este tiene pocos parámetros.

En las siguientes subsecciones se presentan los resultados obtenidos para cada experimento, analizando la evolución del entrenamiento, el rendimiento sobre el conjunto de test y las métricas de clasificación, así como su comparación cualitativa con los resultados obtenidos en el Escenario 1.

4.2.1. Experimento 1: VGG16 con Adam y Batch Size 64

En este experimento se evalúa la arquitectura **VGG16** sobre el conjunto de datos **filtrado completo**, considerando las **15 clases originales** del problema de *Human Action Recognition*. Se utiliza la misma configuración de entrenamiento que mostró el mejor rendimiento en el Escenario 1, con el objetivo de analizar el impacto del aumento del número de clases manteniendo constante tanto el preprocesado de las imágenes como los hiperparámetros principales del entrenamiento.

Este experimento sirve como referencia para evaluar la capacidad de generalización de VGG16 en un escenario de mayor complejidad semántica, así como para comparar posteriormente su comportamiento con arquitecturas más avanzadas.

Configuración del entrenamiento

La configuración empleada en este experimento se resume en la Tabla 4.14. En comparación con el Escenario 1, la principal diferencia radica en el incremento del número de clases de salida, que pasa de 5 a 15, manteniéndose el filtrado por resolución aplicado al conjunto de datos.

Tabla 4.14: Configuración del Experimento 1 del Escenario 2

Modelo	VGG16
Optimizador	Adam
Batch size	64
Learning Rate (Transfer Learning)	1×10^{-4}
Épocas (Transfer Learning)	10
Learning Rate (Fine-tuning)	1×10^{-5}
Épocas (Fine-tuning)	10
Número de clases	15
Número total de parámetros	14.981.957

Evolución del entrenamiento

La Figura 4.13 muestra la evolución de las métricas de *accuracy*, *loss*, *learning rate* y número de parámetros entrenables durante las fases de *transfer learning* y *fine-tuning*.

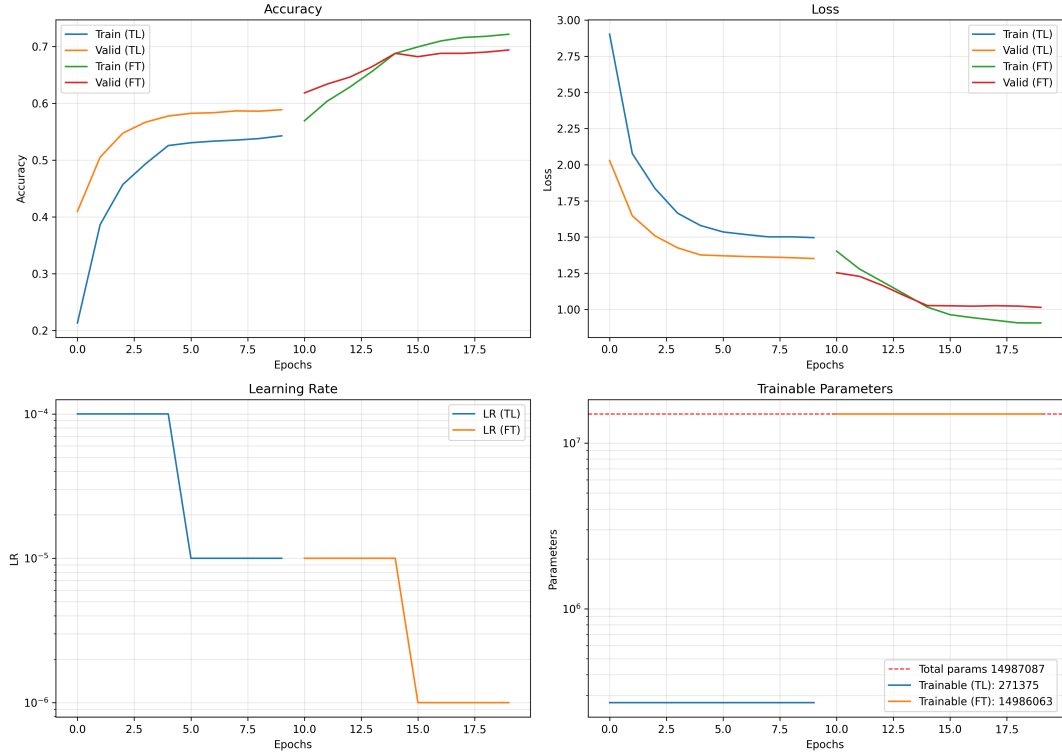


Figura 4.13: Evolución del entrenamiento durante las fases de *transfer learning* y *fine-tuning* — Escenario 2, Experimento 1

Durante la fase de *transfer learning* se observa una convergencia progresiva tanto en entrenamiento como en validación, aunque con valores de *accuracy* claramente inferiores a los obtenidos en el Escenario 1. Este comportamiento es coherente con el incremento de la dificultad del problema al considerar un mayor número de clases.

La transición a la fase de *fine-tuning* permite refinar los pesos de la red completa, mejoran significativamente las métricas de *accuracy* y *loss*.

Análisis de la matriz de confusión

La matriz de confusión obtenida sobre el conjunto de test se presenta en la Figura 4.14.

La matriz de confusión evidencia una mayor dispersión de errores entre clases en comparación con el Escenario 1. Este patrón confirma el aumento de dificultad introducido al trabajar con el conjunto completo de clases, aunque el modelo mantiene un comportamiento coherente sin colapsar hacia clases dominantes.

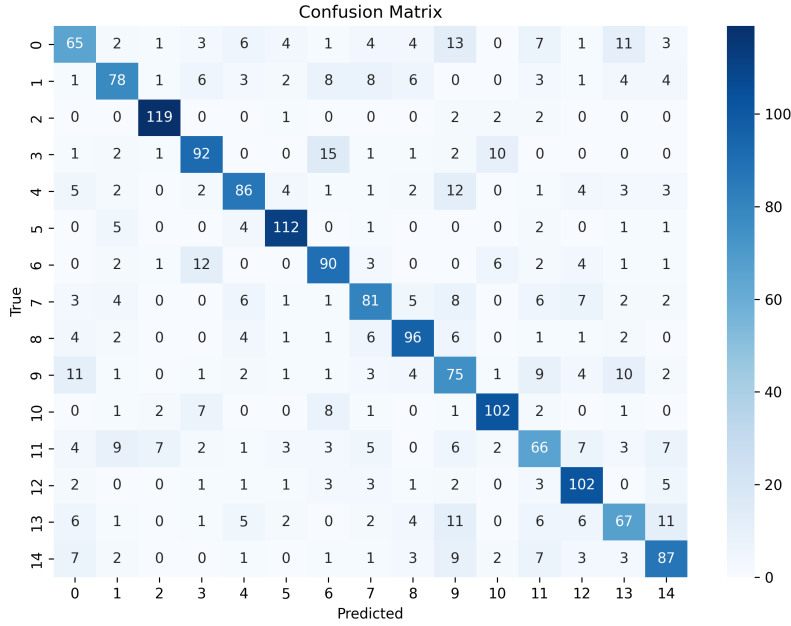


Figura 4.14: Matriz de confusión — Escenario 2, Experimento 1

Resultados finales

Los resultados cuantitativos finales del experimento se recogen en la Tabla 4.15.

Tabla 4.15: Resultados finales del Escenario 2, Experimento 1

Métrica	Train	Validación	Test
Accuracy	0.72	0.69	0.70
Loss	0.91	1.01	—
Precision (macro)	—	—	0.70
Recall (macro)	—	—	0.70
F1-score (macro)	—	—	0.70
Top-3 Accuracy	—	—	0.90
Top-5 Accuracy	—	—	0.95

En conjunto, los resultados muestran una degradación del rendimiento respecto al Escenario 1, como era esperable al incrementar el número de clases, pero confirman que la configuración seleccionada mantiene un comportamiento estable y una capacidad de generalización razonable.

4.2.2. Experimento 2: EfficientNetB3 con Adam y Batch Size 64

En este experimento se evalúa la arquitectura **EfficientNetB3** sobre el conjunto de datos filtrado completo, considerando las **15 clases** del problema de *Human Action Recognition*. El objetivo es analizar si el modificar la arquitectura del modelo mejora o no las métricas del experimento anterior.

Configuración del entrenamiento

La configuración empleada en este experimento se resume en la Tabla 4.16. La principal diferencia respecto al Experimento 1 del Escenario 2 radica en el uso de la arquitectura EfficientNetB3 y en el valor del *learning rate* inicial durante la fase de *transfer learning*.

Tabla 4.16: Configuración del Experimento 2 del Escenario 2

Modelo	EfficientNetB3
Optimizador	Adam
Batch size	64
Learning Rate (Transfer Learning)	1×10^{-4}
Épocas (Transfer Learning)	10
Learning Rate (Fine-tuning)	1×10^{-5}
Épocas (Fine-tuning)	10
Número de clases	15
Número total de parámetros	11.399.469

Evolución del entrenamiento

La Figura 4.15 muestra la evolución de las métricas de entrenamiento y validación durante las fases de *transfer learning* y *fine-tuning*.

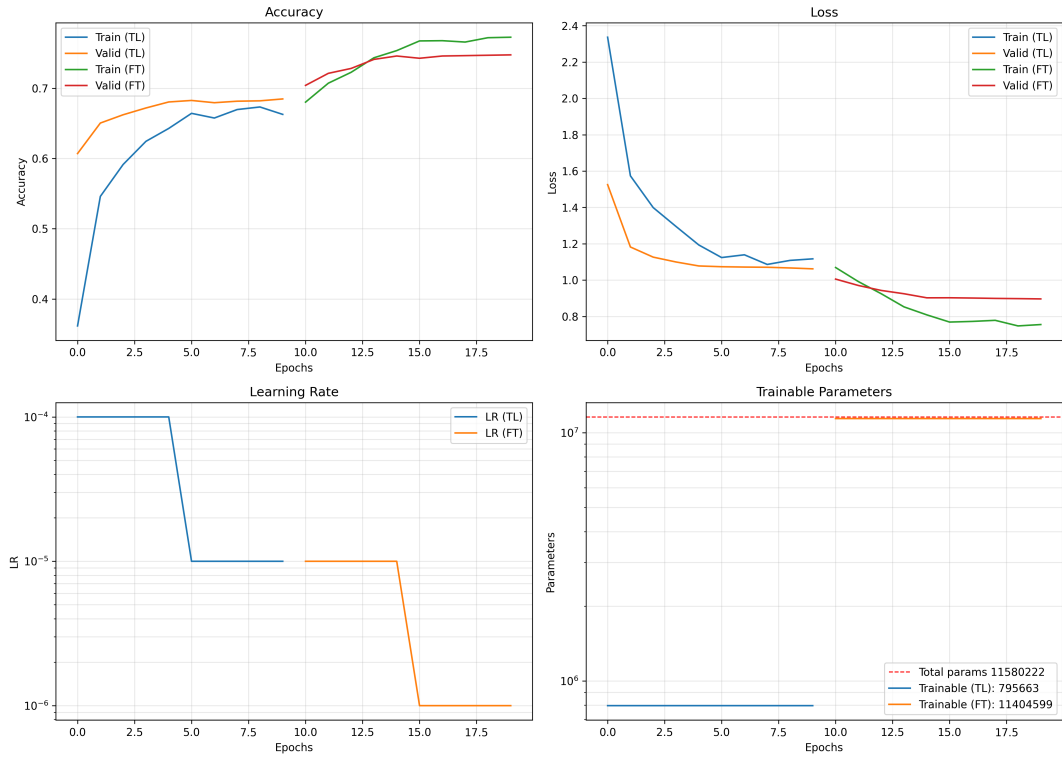


Figura 4.15: Evolución del entrenamiento durante las fases de *transfer learning* y *fine-tuning* — Escenario 2, Experimento 2

Durante la fase de *transfer learning*, el modelo muestra una convergencia progresiva. La transición a la fase de *fine-tuning* permite continuar mejorando el rendimiento, no presentan síntomas de overfitting y mejoran el Experimento 1 del Escenario 2. A pesar de ello, se reafirma al comparar con el Experimento 5 del Escenario 1 que el aumentar el número de clases dificulta la clasificación, a pesar de ello las conclusiones y la performance es buena.

Análisis de la matriz de confusión

La matriz de confusión correspondiente al conjunto de test se presenta en la Figura 4.16.

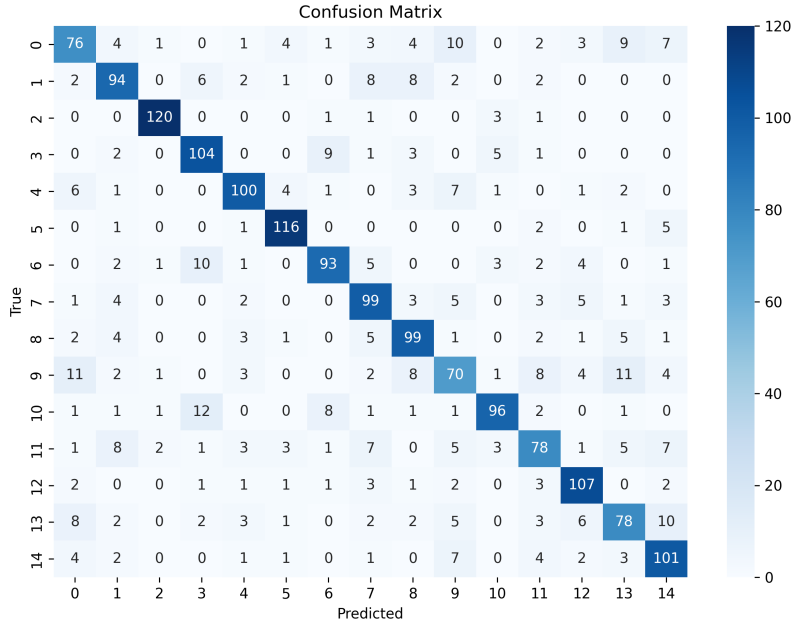


Figura 4.16: Matriz de confusión — Escenario 2, Experimento 2

La matriz de confusión presenta una mejor performance que en el experimento anterior, sin claros síntomas de sesgos.

Resultados finales

Los resultados cuantitativos finales del experimento se recogen en la Tabla 4.17.

Tabla 4.17: Resultados finales del Escenario 2, Experimento 2

Métrica	Train	Validación	Test
Accuracy	0.77	0.74	0.76
Loss	0.76	0.90	—
Precision (macro)	—	—	0.76
Recall (macro)	—	—	0.76
F1-score (macro)	—	—	0.75
Top-3 Accuracy	—	—	0.92
Top-5 Accuracy	—	—	0.96

En comparación con el Experimento 1 del Escenario 2, el uso de EfficientNetB3 permite una mejora moderada en las métricas de clasificación.

4.3. Escenario 3: Entrenamiento sobre el dataset reducido sin filtrar

En este último escenario se analiza el impacto real del proceso de filtrado de imágenes en el rendimiento de los modelos. Para ello, se evalúan dos configuraciones experimentales, correspondientes a las arquitecturas de **VGG16** y **EfficientNetB0**, pero entrenado ahora sobre el **dataset reducido sin aplicar ningún criterio de filtrado** basado en ancho, alto o resolución de las imágenes.

El objetivo principal de este escenario es aislar el efecto del preprocesado y la limpieza de datos, comparando directamente los resultados obtenidos con y sin filtrado, manteniendo constantes tanto la arquitectura del modelo como los hiperparámetros de entrenamiento.

4.3.1. Experimento 1: VGG16 con Adam y Batch Size 64

En este experimento se emplea la arquitectura **VGG16**, utilizando la misma configuración que mostró el mejor rendimiento en el Escenario 1 para esta arquitectura. De este modo, cualquier variación en los resultados puede atribuirse principalmente a la calidad del conjunto de datos, y no a cambios en el modelo o en la estrategia de entrenamiento.

Configuración del entrenamiento

La configuración utilizada en este experimento se resume en la Tabla 4.18.

Tabla 4.18: Configuración del Experimento 1 del Escenario 3

Modelo	VGG16
Optimizador	Adam
Batch size	64
Learning Rate (Transfer Learning)	1×10^{-4}
Épocas (Transfer Learning)	10
Learning Rate (Fine-tuning)	1×10^{-5}
Épocas (Fine-tuning)	10
Número de clases	5
Número total de parámetros	14.981.957

Evolución del entrenamiento

La Figura 4.17 muestra la evolución de las métricas de entrenamiento y validación durante las fases de *transfer learning* y *fine-tuning*.

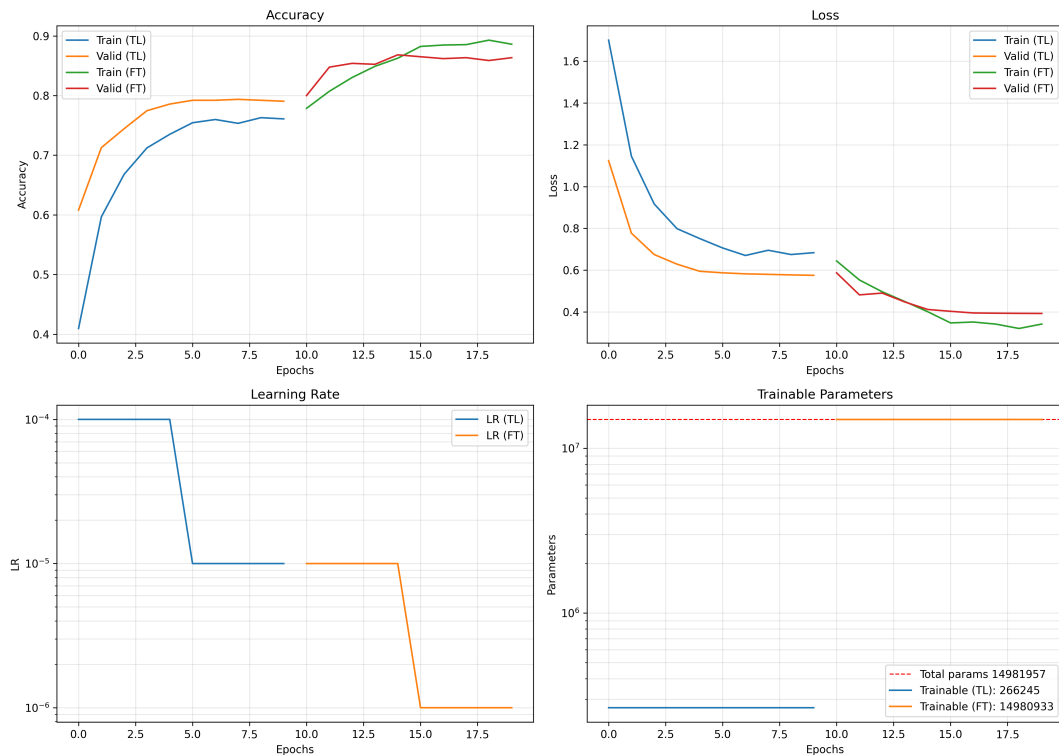


Figura 4.17: Evolución del entrenamiento durante las fases de *transfer learning* y *fine-tuning* — Experimento 1

El entrenamiento muestra un comportamiento estable tanto en entrenamiento como en validación. El modelo alcanza valores elevados de *accuracy* en ambas fases, lo que indica una correcta adaptación al conjunto de datos incluso en ausencia del filtrado previo.

No obstante, al comparar estas curvas con las obtenidas en el Experimento 3 del Escenario 1, se observa una ligera degradación en la convergencia final, lo que sugiere que la presencia de imágenes con resoluciones más heterogéneas introduce cierta dificultad adicional en el proceso de aprendizaje.

Análisis de la matriz de confusión

La matriz de confusión correspondiente al conjunto de test se presenta en la Figura 4.18.

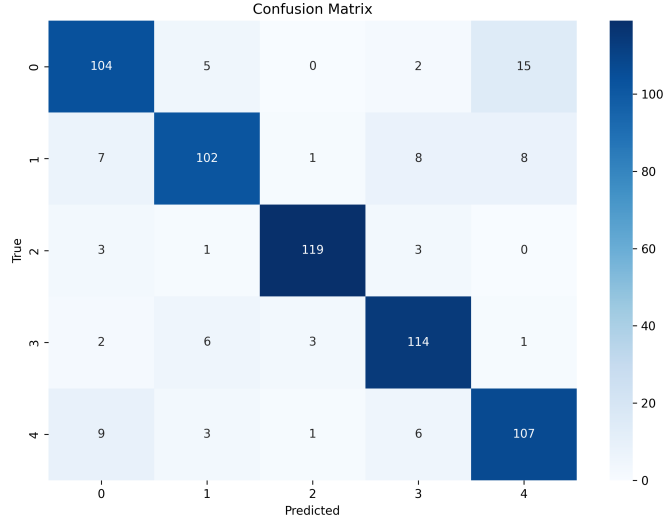


Figura 4.18: Matriz de confusión — Escenario 3, Experimento 1

La matriz de confusión muestra un patrón de errores similar al observado en el Escenario 1, aunque con más errores. Este comportamiento confirma que, si bien el modelo sigue siendo capaz de identificar correctamente la mayoría de las acciones, la ausencia de filtrado incrementa la confusión.

Resultados finales

Los resultados cuantitativos finales del experimento se recogen en la Tabla 4.19.

Tabla 4.19: Resultados finales del Escenario 3, Experimento 1

Métrica	Train	Validación	Test
Accuracy	0.89	0.86	0.87
Loss	0.34	0.39	—
Precision (macro)	—	—	0.87
Recall (macro)	—	—	0.87
F1-score (macro)	—	—	0.87
Top-3 Accuracy	—	—	0.99

En comparación con el Experimento 3 del Escenario 1, los resultados obtenidos en este escenario son ligeramente inferiores, a pesar de utilizar la misma arquitectura y configuración de entrenamiento. Esta diferencia confirma que el proceso de filtrado de las imágenes en función de su ancho, alto y resolución contribuye de forma positiva

a la calidad del conjunto de datos y, en consecuencia, a una mejora del rendimiento y la capacidad de generalización del modelo.

4.3.2. Experimento 2: EfficientNetB0 con Adam y Batch Size 64

En este experimento se emplea la arquitectura **EfficientNetB0**.

Configuración del entrenamiento

La configuración utilizada en este experimento se resume en la Tabla 4.20.

Tabla 4.20: Configuración del Experimento 2 del Escenario 3

Modelo	EfficientNetB0
Optimizador	Adam
Batch size	64
Learning Rate (Transfer Learning)	1×10^{-4}
Épocas (Transfer Learning)	10
Learning Rate (Fine-tuning)	1×10^{-5}
Épocas (Fine-tuning)	10
Número de clases	5
Número total de parámetros	4.710.056

Evolución del entrenamiento

La Figura 4.19 muestra la evolución de las métricas de entrenamiento y validación durante las fases de *transfer learning* y *fine-tuning*.

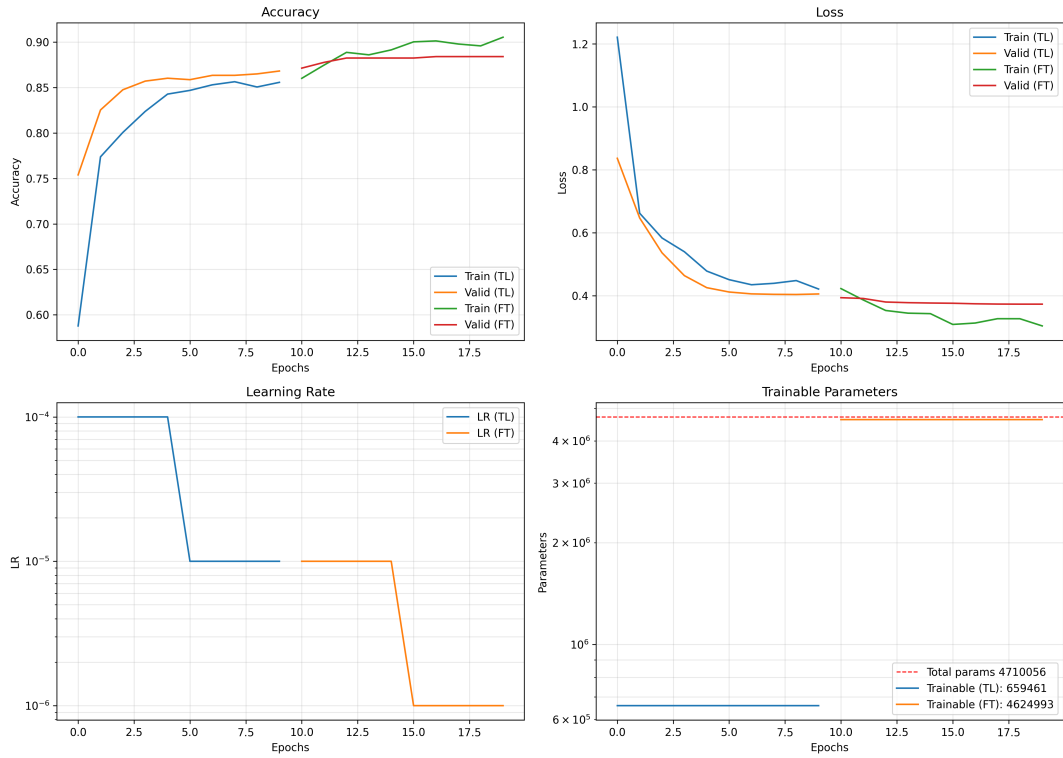


Figura 4.19: Evolución del entrenamiento durante las fases de *transfer learning* y *fine-tuning* — Escenario 3, Experimento 2

El entrenamiento muestra un comportamiento de mejora progresiva tanto en la etapa de *transfer learning* como en *finetuning* e incluso presenta mejoras en comparación con el Experimento 1 del Escenario 3. Con menos parámetros ha conseguido realizar una mejor performance.

Análisis de la matriz de confusión

La matriz de confusión correspondiente al conjunto de test se presenta en la Figura 4.20.

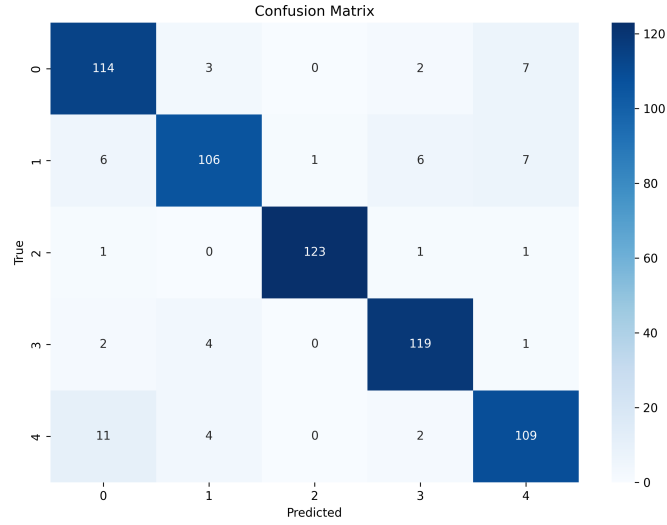


Figura 4.20: Matriz de confusión — Escenario 3, Experimento 2

La matriz de confusión muestra una mejora significativa en comparación con el experimento anterior.

Resultados finales

Los resultados cuantitativos finales del experimento se recogen en la Tabla 4.21.

Tabla 4.21: Resultados finales del Escenario 3, Experimento 2

Métrica	Train	Validación	Test
Accuracy	0.91	0.88	0.91
Loss	0.30	0.37	—
Precision (macro)	—	—	0.91
Recall (macro)	—	—	0.91
F1-score (macro)	—	—	0.91
Top-3 Accuracy	—	—	0.99

Las métricas obtenidas rompen el esquema y las hipótesis obtenidas, ya que se consigue mejores métricas incluso que en el Escenario 1 donde se realizó la limpieza del dataset. El motivo puede estar no solo en la mejora de las capacidades del modelo y sus optimizaciones si no también en su preprocesado, y al contar con un mayor volumen de imágenes (aproximadamente 100 más, que no son muchas) el modelo realiza una mejor performance.

Capítulo 5

Conclusiones

En este trabajo se ha abordado el problema de clasificación de acciones en imágenes a partir del dataset ***Human Action Recognition*** disponible en Hugging Face, compuesto por 15 clases y un volumen de 18.000 imágenes. El desarrollo del proyecto pone a prueba técnicas de visión por computador, donde se realiza un estudio importante del conjunto de los datos, con el posterior estudio del rendimiento de entrenamiento de modelos de clasificación sobre este dataset.

El análisis exploratorio inicial evidenció dos aspectos críticos. En primer lugar, se observó un desequilibrio y un problema de etiquetado en el subconjunto original de test, donde las etiquetas no guardaban correspondencia con el contenido visual. Ante esta situación, se decidió renombrar dicho subconjunto como *noise*, evitando su uso como conjunto de evaluación final y preservándolo como material potencialmente útil en etapas posteriores del proyecto. Como línea de trabajo futura, este subconjunto podría abordarse mediante un enfoque de etiquetado semisupervisado.

En segundo lugar, el dataset presentaba una elevada heterogeneidad en **ancho, alto y resolución** de las imágenes, con presencia de valores atípicos que podían degradar el aprendizaje al introducir reescalados agresivos y pérdida de información visual relevante. Para mitigar este problema, se evaluaron dos estrategias de filtrado: DBSCAN como método de detección de outliers por densidad, y un filtrado determinista basado en rangos intercuartílicos aplicados conjuntamente a dimensiones y resolución. Finalmente, se seleccionó el enfoque basado en **IQR**, ya que ofrece un criterio **reproducible, interpretable y fácil de justificar**, preservando además un porcentaje alto de datos sin eliminar de forma innecesaria muestras válidas.

A nivel metodológico, la estrategia de *transfer learning* seguida de *fine-tuning* ha mostrado ser adecuada para adaptar modelos sobre el dataset ya que no solo se ajustan los pesos de salida del modelo. Este proceso puede deteriorar temporalmente las métricas, pero finalmente las acaba mejorando. Por otro lado, al emplear esta estrategia el callback de EarlyStop debe ser desactivado al menos durante esas primeras etapas donde el modelo empeora.

Del dataset original se crearon tres escenarios: dataset filtrado y reducido seleccionando solo cinco clases, dataset filtrado y con todas las clases y dataset reducido pero sin filtrar. Se puso más incapié en el primer escenario donde se probó diferentes arquitecturas (VGG16, VGG19 EfficientNetB0 y EfficientNetB3), diferentes optimizadores (SGD Y Adam) y la variación de Batch Size.

A nivel metodológico, la estrategia de *transfer learning* seguida de *fine-tuning* ha mostrado ser adecuada para adaptar modelos al dominio del dataset, ya que no solo se ajustan las capas finales, sino que se refinan también las representaciones convolucionales. Este proceso puede deteriorar temporalmente las métricas al inicio del *fine-tuning*, pero termina traducándose en mejoras sostenidas en *accuracy* y en la función de pérdida. En este contexto, resulta recomendable desactivar *Early Stopping* (o relajar su criterio) durante la transición entre fases, evitando detener el entrenamiento de forma prematura en el periodo de reajuste de pesos.

Del dataset original se definieron tres escenarios experimentales: (i) dataset reducido y filtrado seleccionando cinco clases, (ii) dataset filtrado completo con las quince clases, y (iii) dataset reducido sin filtrado. Se puso especial énfasis en el primer escenario, donde se exploraron distintas arquitecturas (VGG16, VGG19, EfficientNetB0 y EfficientNetB3), diferentes optimizadores (SGD y Adam) y la variación del *batch size*. Esta exploración permitió identificar configuraciones robustas y extraer conclusiones sobre el impacto relativo de la calidad del dato, la complejidad del problema y las decisiones de diseño del entrenamiento.

En el **Escenario 1** (dataset reducido y filtrado, 5 clases) se realizó la exploración principal de configuraciones. En primer lugar, se comparó **VGG16** con dos optimizadores: **Adam** y **SGD**, manteniendo el resto de hiperparámetros constantes. Los resultados mostraron que **Adam** proporcionó una convergencia más rápida y estable, logrando métricas claramente superiores, mientras que **SGD** evidenció un comportamiento de *underfitting* incluso aumentando el número de épocas. Posteriormente, al **incrementar el batch size** (de 8 a 64) con VGG16+Adam se observó

una mejora en la estabilidad del entrenamiento y, especialmente, en la **capacidad de generalización**, reflejada en un aumento del rendimiento en test.

A continuación, se evaluó el efecto del cambio de arquitectura. Con **EfficientNetB0**, pese a contar con un número de parámetros muy inferior a VGG, se alcanzó un rendimiento igual o mejor y un entrenamiento particularmente estable, lo que indica un mejor compromiso entre eficiencia y capacidad representacional para este problema. Sin embargo, al escalar a arquitecturas más complejas (**EfficientNetB3** y **VGG19**), la ganancia fue marginal o inexistente respecto a modelos más ligeros, por lo que **no se justificó el incremento de coste computacional**. En conjunto, este escenario sugiere que, bajo condiciones de datos controladas, **la elección del optimizador y el tamaño de batch influyen de forma determinante**, y que modelos moderados como EfficientNetB0 pueden ser preferibles frente a alternativas más grandes.

En el **Escenario 2** (dataset completo filtrado, 15 clases) se mantuvo el filtrado y se incrementó la dificultad del problema. Se evaluaron **VGG16** y **EfficientNetB3** (ambos con un número de parámetros elevado y comparable), observándose una **buena convergencia** y ausencia de patrones claros de colapso o sesgo, pero una **degradación esperable de las métricas** debido al aumento del número de clases. Aun así, EfficientNetB3 mantuvo un rendimiento sólido, alcanzando valores en torno al **0.76–0.77** en *accuracy/precision/recall* macro, lo que confirma una capacidad de generalización razonable en un escenario semánticamente más exigente.

Por último, en el **Escenario 3** (dataset reducido sin filtrar) se aisló el impacto del filtrado de imágenes. Aunque se esperaba una degradación sistemática al introducir resoluciones y proporciones más heterogéneas, **EfficientNetB0** presentó una **mejora inesperada** y resultados incluso superiores al escenario filtrado. Este comportamiento puede atribuirse a la **robustez del preprocesado implícito del modelo**, a su diseño arquitectónico y a las optimizaciones internas que facilitan la adaptación a variabilidad visual, además de un ligero incremento de datos disponibles. En consecuencia, aunque el filtrado por IQR resulta justificable y beneficioso en términos de control y consistencia del dataset, los resultados sugieren que **arquitecturas modernas y eficientes pueden compensar parte de la heterogeneidad** y, en ciertos casos, incluso beneficiarse de la diversidad adicional del conjunto sin filtrar.

Bibliografía

- [1] Hugging face datasets hub. <https://huggingface.co/datasets>, 2026. Accedido: 2026-01-03.
- [2] Haikel Alhichri, Asma Alsuwayed, Yakoub Bazi, Nassim Ammour, and Naif Alajlan. Classification of remote sensing images using efficientnet-b3 cnn model with attention. *IEEE Access*, PP:1–1, 01 2021.
- [3] Hugging Face Datasets. Bingsu/human_action_recognition dataset. https://huggingface.co/datasets/Bingsu/Human_Action_Recognition, 2022. Accedido: 03 Enero 2026.
- [4] Kamran Khan, Saif Ur Rehman, Kamran Aziz, Simon Fong, and S. Sarasvady. Dbscan: Past, present and future. In *The Fifth International Conference on the Applications of Digital Information and Web Technologies (ICADIWT 2014)*, pages 232–238, 2014.
- [5] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [6] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. *arXiv preprint arXiv:1312.4400*, 2013.
- [7] H. Robbins and S. Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [8] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- [9] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. *arXiv preprint arXiv:1905.11946*, 2019.